

ĐẠI HỌC QUỐC GIA TP.HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH



BÁO CÁO ĐỒ ÁN
Vi xử lý, vi điều khiển - CE103.Q21

ĐỀ TÀI
Bảng tin Bluetooth di động

Giảng viên hướng dẫn: TS. Phạm Quốc Hùng

Sinh viên thực hiện:

Lâm Vương Quốc Lĩnh - 24520957

Lê Đức Nhân - 24521231

Lê Phúc Tài - 24521548

TP. Hồ Chí Minh, Tháng 5 năm 2026



LỜI MỞ ĐẦU

Ngày nay, với sự phát triển không ngừng của cuộc cách mạng công nghiệp 4.0, cuộc sống của chúng ta đã được nâng cao đáng kể nhờ vào các thiết bị và hệ thống hiện đại phục vụ cho quá trình công nghiệp hóa– hiện đại hóa. Đóng góp vào sự tiến bộ đó không thể không kể đến các hệ thống nhúng và công nghệ internet vạn vật. Từ những ngày đầu ra đời, các vi xử lý và vi điều khiển đã chứng tỏ được tính ưu việt của chúng, và cho đến nay, những thành tựu này càng được khẳng định mạnh mẽ hơn. Những ứng dụng của chúng đã biến những điều tưởng chừng như không thể trở thành hiện thực, góp phần cải thiện và nâng cao chất lượng cuộc sống của con người.

Nhằm minh chứng cho hiệu quả thực tế của các ứng dụng trong môn học vi xử lý– vi điều khiển, sau một thời gian học tập và nghiên cứu, dưới sự hướng dẫn tận tình của thầy cô và sự nỗ lực không ngừng của bản thân, chúng em đã thực hiện đề án "Bảng tin Bluetooth di động". Dự án này nhằm mục đích ứng dụng các kiến thức đã học vào việc thiết kế một hệ thống hiển thị tin nhắn được lưu trong bộ nhớ bằng cách gửi từ các thiết bị di động có hỗ trợ kết nối Bluetooth, cũng như vận dụng sự sáng tạo để nâng cao chất lượng của đồ án trong quá trình trải nghiệm đến từ người dùng, đạt chất lượng và kết quả tốt nhất.

Do hạn chế về thời gian, kiến thức và kinh nghiệm, chắc chắn đề án của chúng em sẽ không tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự góp ý, hỗ trợ từ thầy cô và các bạn để có thể hoàn thiện đề án này.

TP. Hồ Chí Minh, tháng 05 năm 2026

Nhóm sinh viên thực hiện

Lâm Vương Quốc Lĩnh

Lê Đức Nhân

Lê Phúc Tài



LỜI CẢM ƠN

Trước tiên, chúng em xin bày tỏ lòng biết ơn sâu sắc đến thầy Phạm Quốc Hùng, giảng viên bộ môn Vi xử lý – Vi điều khiển, Khoa Kỹ thuật Máy tính, Trường Đại học Công nghệ Thông tin.

Chính nhờ sự hướng dẫn tận tình, chỉ bảo chu đáo và những góp ý quý báu từ thầy trong suốt quá trình học tập và thực hiện bài báo cáo, chúng em mới có thể hoàn thành tốt nhiệm vụ được giao.

Quãng thời gian học tập cùng thầy đã giúp chúng em không chỉ tiếp thu thêm nhiều kiến thức chuyên môn bổ ích mà còn học hỏi được những kinh nghiệm thực tiễn giá trị. Đồng thời, thầy đã giúp chúng em rèn luyện các kỹ năng làm việc nhóm, tư duy logic, và giải quyết vấn đề – những hành trang quan trọng để chúng em phát triển bản thân và vững bước trong tương lai.

Chúng em xin chân thành cảm ơn thầy đã luôn nhiệt huyết, tạo động lực để chúng em nỗ lực hoàn thiện bài báo cáo này.

Môn học Vi xử lý – Vi điều khiển không chỉ mang lại cho chúng em những kiến thức lý thuyết hữu ích mà còn là cơ hội để vận dụng và thực hành, giúp chúng em hiểu sâu hơn về lĩnh vực mình đang theo đuổi. Với sự cố gắng của cả nhóm cùng sự hỗ trợ từ thầy, bài báo cáo đã được hoàn thành đúng thời hạn.

Tuy nhiên, do kiến thức và kinh nghiệm thực tế của nhóm còn nhiều hạn chế, bài báo cáo không tránh khỏi những thiếu sót. Chúng em kính mong thầy xem xét và đóng góp ý kiến để bài làm của nhóm được hoàn thiện hơn.

Một lần nữa, chúng em xin gửi lời tri ân chân thành đến thầy và kính chúc thầy dồi dào sức khỏe, thành công trong sự nghiệp giảng dạy.

TP. Hồ Chí Minh, tháng 05 năm 2026

Nhóm sinh viên thực hiện

Lâm Vương Quốc Linh

Lê Đức Nhân

Lê Phúc Tài



Mục lục

DANH MỤC HÌNH ẢNH	5
DANH MỤC BẢNG BIỂU	6
1 TỔNG QUAN ĐỒ ÁN	7
1.1 Lý do chọn đề tài	7
1.2 Mục tiêu đề tài	7
1.3 Nội dung thực hiện	7
1.4 Phương pháp thực hiện	8
1.5 Giới hạn đề tài và kết quả mong đợi	8
2 CƠ SỞ LÝ THUYẾT	10
2.1 Vi điều khiển ESP32	10
2.2 Các giao thức truyền thông	10
2.2.1 Giao thức SPI (giao tiếp TFT)	10
2.2.2 Giao thức I2S (giao tiếp loa)	10
2.2.3 Giao thức UART/Bluetooth Serial (giao tiếp với điện thoại)	11
2.3 Công nghệ Bluetooth	11
2.3.1 Bluetooth Classic và BLE	11
2.3.2 Serial Port Profile (SPP)	12
2.4 Công nghệ WiFi	12
2.4.1 Chuẩn 802.11	12
2.4.2 Kết nối Internet và HTTP Client	12
2.5 Màn hình TFT	13
2.5.1 Cấu tạo, nguyên lý hiển thị	13
2.5.2 Điều khiển bằng thư viện TFT_eSPI	13
2.6 Quy trình xử lý âm thanh kỹ thuật số	13
2.6.1 Chuẩn nén âm thanh MP3 và Google TTS API	13
2.6.2 Cơ chế giải mã và bộ nhớ đệm (Buffering)	14
2.6.3 Mạch khuếch đại và đầu ra âm thanh	14
2.7 Bộ nhớ Flash (Preferences)	14
2.7.1 Lưu trữ dữ liệu không bay hơi	14
2.7.2 Quản lý tin nhắn trên ESP32	14
3 CÁC PHẦN TỬ, LINH KIỆN TRONG ĐỒ ÁN	15
3.1 Sơ đồ khối	15
3.2 DevBoard ESP32 WROOM 32U CP2102	16
3.2.1 Tổng quan	16
3.2.2 Thông số kỹ thuật	16
3.2.3 Hình ảnh, mô tả chân	17
3.3 Module Bluetooth HC-05	18
3.3.1 Tổng quan	18
3.3.2 Thông số kỹ thuật	19



3.3.3	Hình ảnh, mô tả chân	19
3.4	Mạch khuếch đại âm thanh Max98357	20
3.4.1	Tổng quan	20
3.4.2	Thông số kỹ thuật	20
3.4.3	Hình ảnh, mô tả chân	21
3.5	Loa 3W	21
3.6	Màn hình TFT	21
3.6.1	Tổng quan	21
3.6.2	Hình ảnh, mô tả chân	22
3.7	Button	23
3.7.1	Sơ đồ chân	23
4	THIẾT KẾ HỆ THỐNG	24
4.1	Sơ đồ kết nối chân	24
4.2	Lưu đồ thuật toán	24
4.2.1	Các biến được khởi tạo	24
4.2.2	Hàm kiểm tra chuỗi có phải là số hay không (<code>isNumber()</code>)	27
4.2.3	Hàm lấy tên chế độ hiển thị (<code>getModeName()</code>)	28
4.2.4	Hàm cập nhật giao diện hiển thị (<code>updateUI()</code>)	29
4.2.5	Hàm tải tin nhắn từ bộ nhớ (<code>loadMessage()</code>)	30
4.2.6	Hàm xóa bộ nhớ flash (<code>clearFlash()</code>)	31
4.2.7	Hàm xử lý nút nhấn (<code>handleButton()</code>)	32
4.2.8	Hàm xử lý yêu cầu phát âm thanh (<code>handleSpeakRequest()</code>)	33
4.2.9	Hàm xử lý hiệu ứng hiển thị (<code>handleAnimation()</code>)	33
4.2.10	Hàm mã hóa chuỗi URL (<code>urlencode()</code>)	34
4.2.11	Hàm phát tiếng Việt (<code>speakVietnamese()</code>)	35
4.3	Code và giải thích thuật toán	36
4.4	Hiện thực phần cứng (video, hình ảnh)	45
5	KẾT LUẬN	46
5.1	Kết quả đã đạt được:	46
5.2	Hạn chế:	46
5.3	Hướng phát triển:	47
	TÀI LIỆU THAM KHẢO	48



DANH MỤC HÌNH ẢNH

1	Sơ đồ khối của hệ thống	15
2	Sơ đồ chân của ESP32	17
3	Module Bluetooth HC-05	19
4	Sơ đồ chân của module khuếch đại Max98357	21
5	Sơ đồ chân của màn hình TFT	22
6	Sơ đồ chân của nút nhấn	23
7	Sơ đồ kết nối chân của hệ thống	24
8	Lưu đồ thuật toán của hàm <code>isNumber()</code>	27
9	Lưu đồ thuật toán của hàm <code>getModeName()</code>	28
10	Lưu đồ thuật toán của hàm <code>updateUI()</code>	29
11	Lưu đồ thuật toán của hàm <code>loadMessage()</code>	30
12	Lưu đồ thuật toán của hàm <code>clearFlash()</code>	31
13	Lưu đồ thuật toán của hàm <code>handleButton()</code>	32
14	Lưu đồ thuật toán của hàm <code>handleSpeakRequest()</code>	33
15	Lưu đồ thuật toán của hàm <code>handleAnimation()</code>	33
16	Lưu đồ thuật toán của hàm <code>urlencode()</code>	34
17	Lưu đồ thuật toán của hàm <code>speakVietnamese()</code>	35
18	Lưu đồ thuật toán tổng quát của hệ thống	44
19	Hình ảnh mạch thực tế	45



DANH MỤC BẢNG BIỂU

1	Mô tả các chân tín hiệu của ESP32	18
2	Mô tả các chân tín hiệu của module Bluetooth HC-05	20
3	Mô tả các chân tín hiệu của module MAX98357	21
4	Mô tả các chân tín hiệu của màn hình TFT	22
5	Các hằng số và biến cấu hình chính của hệ thống	25
6	Các biến thời gian của hệ thống	26
7	Các biến logic và trạng thái của hệ thống	26
8	Các biến chuỗi và đối tượng được khởi tạo	26



1 TỔNG QUAN ĐỒ ÁN

1.1 Lý do chọn đề tài

Trong thời đại chuyển đổi số, việc truyền tải thông tin cần sự nhanh chóng và tiện lợi. Các bảng tin truyền thống thường cố định nội dung hoặc đòi hỏi kết nối dây phức tạp để thay đổi thông tin. Đề tài này giải quyết bài toán đó bằng cách sử dụng kết nối không dây Bluetooth, cho phép người dùng cập nhật nội dung tức thời từ điện thoại cá nhân. Đây là giải pháp tối ưu cho các môi trường năng động như cửa hàng bán lẻ, văn phòng hoặc các biển báo điều hướng tạm thời nhờ tính di động cao và khả năng tùy biến nội dung linh hoạt.

Trong đề tài này, thay vì sử dụng các thư viện tích hợp sẵn, việc lựa chọn giao tiếp UART vật lý giữa module Bluetooth rời và vi điều khiển. Bên cạnh đó, việc xử lý thuật toán dịch bit để tạo hiệu ứng chạy chữ trên màn hình TFT và quản lý bộ nhớ Non-Volatile (Flash/EEPROM) giúp củng cố kiến thức về tối ưu hóa tài nguyên và đảm bảo tính toàn vẹn dữ liệu của hệ thống những khi xảy ra sự cố mất điện.

Đề tài **Bảng tin Bluetooth di động** không chỉ dừng lại ở mức độ mô hình thí nghiệm mà còn sở hữu tiềm năng phát triển thành sản phẩm thương mại hoàn thiện. Với nền tảng phần cứng là ESP - 32, hệ thống có thể dễ dàng nâng cấp để kết nối Wi-Fi (IoT) nhằm quản lý từ xa qua Internet, hoặc tích hợp thêm các cảm biến môi trường như nhiệt độ, độ ẩm để biến bảng tin thành một trạm quan trắc thông minh. Việc thiết kế một hệ thống tối ưu giữa tiêu thụ năng lượng và hiệu suất hiển thị chính là bước đệm quan trọng để phát triển các thiết bị điện tử đeo tay hoặc thiết bị gia dụng thông minh trong tương lai.

1.2 Mục tiêu đề tài

Đề tài "**Bảng tin Bluetooth di động**" nhằm mục tiêu thiết kế và triển khai hệ thống bảng tin di động sử dụng ESP32, nhận nội dung qua Bluetooth, hiển thị lên màn hình TFT, lưu tin nhắn và hỗ trợ phát giọng đọc tiếng Việt, cụ thể như sau:

- Nhận chuỗi ký tự hiển thị từ ứng dụng điện thoại qua module Bluetooth (UART).
- Lưu trữ nội dung tin nhắn vào bộ nhớ Flash/EEPROM để hiển thị sau khi mất điện.
- Đổi hiệu ứng hiển thị tin nhắn đa dạng trên màn hình TFT (Chớp nháy, chạy trái/phải).
- Đọc tin nhắn tiếng Việt.
- Xem tin nhắn cũ, khóa thiết bị (read - only mode).
- Nút xóa toàn bộ tin nhắn đã lưu.
- Kết nối WiFi để lấy giọng đọc.

1.3 Nội dung thực hiện

Để hoàn thành đề tài, các nội dung chính cần thực hiện được triển khai theo từng nhóm công việc cụ thể như sau:



- Xây dựng thuật toán điều khiển và xử lý dữ liệu phù hợp với yêu cầu vận hành của hệ thống, bảo đảm tính ổn định, khả năng đáp ứng đối với thực tế.
- Tìm hiểu phương án kết nối phần cứng giữa vi điều khiển với các khối chức năng như module Bluetooth, màn hình TFT, mạch khuếch đại âm thanh, nút nhấn và nguồn cấp đồng thời lựa chọn phương án cấp nguồn phù hợp để hệ thống hoạt động an toàn, liên tục.
- Ứng dụng kiến thức đã học trong học phần Vi xử lý – Vi điều khiển để thiết kế chương trình điều khiển, cấu hình các chuẩn giao tiếp cơ bản, tổ chức lưu trữ dữ liệu và tối ưu tài nguyên phần cứng.
- Phân tích kết quả thực hiện, đánh giá những ưu điểm, hạn chế của mô hình và đề xuất hướng phát triển trong tương lai.

1.4 Phương pháp thực hiện

- Thiết kế sơ đồ hệ thống.
- Sử dụng vi điều khiển ESP32 được thiết kế trên board ESP32 WROOM 32U CP2102, ngôn ngữ C++ trên nền tảng Framework Arduino, sử dụng trình biên dịch dành cho kiến trúc chip ESP32.
- sử dụng phương pháp quét trạng thái nút nhấn kết hợp chống dội bằng thời gian (dành cho đổi chế độ hiển thị, read - only mode).
- Ý tưởng hóa đồ án, chạy thực tế.

1.5 Giới hạn đề tài và kết quả mong đợi

Giới hạn đề tài: Do hạn chế về thời gian nghiên cứu, kinh phí cũng như đặc tính vật lý của các linh kiện phần cứng, đồ án có một số giới hạn nhất định như sau:

- **Khoảng cách giao tiếp:** Hệ thống sử dụng module Bluetooth HC-05 hoạt động theo chuẩn Bluetooth Classic (giao thức SPP), do đó phạm vi kết nối và nhận tin nhắn ổn định chỉ nằm trong bán kính khoảng 10 mét (trong điều kiện lý tưởng, không có vật cản).
- **Khả năng tương thích nền tảng:** Giao thức SPP của module HC-05 hiện tại chủ yếu tương thích tốt với các thiết bị Android thông qua các ứng dụng Bluetooth Terminal. Hệ thống chưa hỗ trợ kết nối trực tiếp với các thiết bị iOS (iPhone/iPad).
- **Nội dung và không gian hiển thị:** Màn hình TFT có kích thước và độ phân giải giới hạn, chủ yếu phục vụ hiển thị văn bản (Text) ngắn gọn. Đề tài chưa hỗ trợ hiển thị các hình ảnh, video đồ họa phức tạp hay các định dạng font chữ cầu kỳ.
- **Điều kiện hoạt động của tính năng giọng nói (TTS):** Mặc dù hệ thống nhận tin nhắn offline qua Bluetooth, nhưng tính năng Text-to-Speech (chuyển văn bản thành giọng nói tiếng Việt) yêu cầu vi điều khiển ESP32 phải được kết nối với mạng WiFi có Internet để gọi API từ Google và tải tệp MP3 về giải mã.



Kết quả mong đợi: Dựa trên các mục tiêu đã đề ra, nhóm nghiên cứu kỳ vọng đạt được những kết quả sau:

- **Về phần cứng:** Thiết kế và lắp ráp thành công mô hình bảng tin vật lý nhỏ gọn, mang tính di động cao. Hệ thống hoạt động ổn định thông qua mạch nguồn ngoài cấp điện trực tiếp lên các thanh ray của testboard, đảm bảo cung cấp năng lượng liên tục và đủ dòng cho cả các tác vụ hiển thị màn hình và phát âm thanh I2S mà không làm quá tải vi điều khiển.
- **Về truyền thông và lưu trữ dữ liệu:** Thiết lập thành công luồng truyền tải dữ liệu nối tiếp qua Bluetooth từ điện thoại đến ESP32 mà không bị mất hoặc sai lệch ký tự. Việc lưu trữ dữ liệu vào bộ nhớ Flash không bay hơi (thông qua thư viện **Preferences**) hoạt động chính xác, bảo toàn được nội dung tin nhắn khi ngắt nguồn điện.
- **Về tính năng và trải nghiệm người dùng:**
 - Hệ thống giải quyết triệt để vấn đề lỗi phong chữ ngôn ngữ bản địa bằng cách tích hợp bộ font mượt *Smooth Font* (Arial 24pt), hiển thị giao diện và nội dung tin nhắn bằng Tiếng Việt có dấu hoàn chỉnh, rõ nét.
 - Các hiệu ứng chữ (tĩnh, chạy chữ, nhấp nháy) vận hành mượt mà trên bộ đệm **TFT_eSprite** giúp loại bỏ hoàn toàn hiện tượng chớp nháy màn hình (flicker).
 - Hệ thống phản hồi âm thanh thông báo và đọc nội dung tin nhắn bằng tiếng Việt lưu loát thông qua Google TTS API.



2 CƠ SỞ LÝ THUYẾT

2.1 Vi điều khiển ESP32

ESP32 là vi điều khiển 32-bit tích hợp sẵn Wi-Fi, Bluetooth Classic và Bluetooth Low Energy (BLE), phù hợp cho nhiều ứng dụng nhúng và Internet vạn vật (IoT). Nhờ hiệu năng tốt, mức tiêu thụ điện năng hợp lý và khả năng kết nối đa dạng, ESP32 được sử dụng rộng rãi trong các hệ thống điều khiển, giám sát và giao tiếp không dây [2, 3].

Một số tính năng nổi bật của ESP-WROOM-32 gồm:

- Sử dụng bộ vi xử lý Tensilica Xtensa[®] Dual-Core 32-bit LX6, xung nhịp tối đa 240 MHz.
- Hiệu năng đạt khoảng 994.26 CoreMark, tương đương 4.14 CoreMark/MHz.
- Tích hợp 448 KB ROM phục vụ quá trình khởi động và các chức năng lõi.
- Tích hợp 520 KB SRAM trên chip để lưu trữ dữ liệu và chương trình.
- Trang bị 4 MB bộ nhớ Flash để lưu mã chương trình và dữ liệu.
- Có thêm 16 KB SRAM trong miền RTC phục vụ các tác vụ tiết kiệm năng lượng.
- Hỗ trợ chuẩn Wi-Fi 802.11b/g/n.
- Hỗ trợ Bluetooth v4.2 BR/EDR và Bluetooth Low Energy (BLE).

2.2 Các giao thức truyền thông

2.2.1 Giao thức SPI (giao tiếp TFT)

SPI (Serial Peripheral Interface) là giao thức truyền thông nối tiếp đồng bộ tốc độ cao, hoạt động theo mô hình Master-Slave và thường dùng khi cần trao đổi dữ liệu nhanh giữa vi điều khiển với thiết bị ngoại vi. Trong đề tài này, SPI được sử dụng để giao tiếp giữa ESP32 và màn hình TFT nhằm bảo đảm tốc độ cập nhật hiển thị.

SPI thường sử dụng bốn đường tín hiệu chính gồm MOSI, MISO, SCK và CS/SS. Trong đó, Master tạo xung clock trên chân SCK, truyền dữ liệu qua MOSI, nhận dữ liệu qua MISO và dùng chân CS để chọn thiết bị cần giao tiếp. Giao thức này hỗ trợ truyền song công toàn phần, tốc độ cao và có thể cấu hình qua hai tham số CPOL, CPHA để phù hợp với từng ngoại vi.

Nhờ cấu trúc đơn giản và khả năng truyền dữ liệu nhanh, SPI rất phù hợp cho các thiết bị cần cập nhật liên tục như bộ nhớ Flash, thẻ SD và màn hình TFT. Vì vậy, đây là lựa chọn phù hợp cho hệ thống hiển thị trong đề tài.

2.2.2 Giao thức I2S (giao tiếp loa)

I2S (Inter-IC Sound) là một chuẩn giao tiếp bus chuyên dụng được thiết kế để truyền tải dữ liệu âm thanh số giữa các vi xử lý và các chip giải mã hoặc khuếch đại âm thanh. Khác với các giao tiếp thông thường, I2S truyền tải dữ liệu âm thanh dưới dạng mã nhị phân PCM một cách liên tục, giúp giảm thiểu tối đa hiện tượng nhiễu và sai số thời gian (jitter). Giao diện I2S trên ESP32 bao gồm 3 đường tín hiệu chính:



- **BCLK (Bit Clock Line):** Xung nhịp điều khiển tốc độ truyền từng bit dữ liệu.
- **LRCK / WS (Word Select):** Tín hiệu xác định dữ liệu thuộc kênh trái (Left) hay kênh phải (Right).
- **DIN (Data Line):** Đường truyền dữ liệu âm thanh dưới dạng mã nhị phân.

Việc sử dụng I2S thay vì các chân Analog hay PWM thông thường giúp hệ thống băng tin đạt được chất lượng âm thanh trung thực và ổn định hơn [4].

2.2.3 Giao thức UART/Bluetooth Serial (giao tiếp với điện thoại)

UART (Universal Asynchronous Receiver/Transmitter) là giao thức truyền thông nối tiếp bất đồng bộ, trong đó hai thiết bị không dùng chung xung clock mà phải thống nhất trước các tham số như tốc độ baud, số bit dữ liệu, parity và stop bit. Nhờ cấu trúc đơn giản, dễ triển khai và được hỗ trợ rộng rãi trên vi điều khiển, UART thường được dùng để giao tiếp với ngoại vi, cấu hình thiết bị và gỡ lỗi hệ thống.

Về phần cứng, UART thường sử dụng hai đường tín hiệu chính là TX và RX cùng với GND. Dữ liệu được truyền theo từng khung gồm start bit, các bit dữ liệu, parity bit (nếu có) và stop bit. Khi hai bên cấu hình không đồng nhất hoặc sai lệch baud rate quá lớn, dữ liệu nhận được có thể bị lỗi.

UART phù hợp với các ứng dụng cần giao tiếp điểm–điểm nhờ chi phí triển khai thấp và cách sử dụng đơn giản. Trong đề tài này, UART được sử dụng gián tiếp thông qua Bluetooth Serial để trao đổi dữ liệu giữa ESP32 và điện thoại, giúp việc gửi tin nhắn và nhận lệnh điều khiển trở nên thuận tiện hơn.

2.3 Công nghệ Bluetooth

2.3.1 Bluetooth Classic và BLE

Bluetooth là công nghệ truyền thông không dây tầm ngắn hoạt động chủ yếu trong băng tần 2.4 GHz, được thiết kế để kết nối các thiết bị ở khoảng cách gần với mức tiêu thụ năng lượng tương đối thấp. Trên ESP32, bộ thu phát Bluetooth hỗ trợ đồng thời Bluetooth Classic (BR/EDR) và Bluetooth Low Energy (BLE), nhờ đó hệ thống có thể linh hoạt lựa chọn phương thức giao tiếp phù hợp với từng yêu cầu ứng dụng.

Bluetooth Classic phù hợp với các bài toán cần duy trì luồng dữ liệu liên tục hoặc tương thích với những profile truyền thống như truyền âm thanh hay cổng nối tiếp ảo. Trong khi đó, BLE được tối ưu cho các ứng dụng cảm biến, thiết bị đeo và hệ thống IoT nhờ cơ chế truyền theo gói ngắn, thời gian quảng bá linh hoạt và mức tiêu thụ năng lượng thấp hơn. Vì vậy, BLE thường được ưu tiên khi cần tiết kiệm pin, còn Bluetooth Classic phù hợp hơn khi cần giao tiếp ổn định và dễ tương thích với điện thoại hoặc máy tính.

Đối với đề tài này, việc trao đổi tin nhắn giữa ESP32 và điện thoại không yêu cầu băng thông lớn nhưng cần cách kết nối đơn giản và dễ triển khai. Do đó, Bluetooth Classic là lựa chọn phù hợp hơn vì cho phép mô phỏng giao tiếp nối tiếp quen thuộc, giúp quá trình gửi và nhận dữ liệu văn bản diễn ra thuận tiện.



2.3.2 Serial Port Profile (SPP)

Serial Port Profile (SPP) là profile của Bluetooth Classic cho phép tạo một kênh truyền dữ liệu có cách sử dụng gần giống cổng UART truyền thống. Khi dùng SPP, dữ liệu giữa hai thiết bị được trao đổi dưới dạng luồng byte, nhờ đó chương trình trên vi điều khiển có thể xử lý thông tin gửi đến và phản hồi theo cách tương tự như giao tiếp nối tiếp thông thường.

Ưu điểm của SPP là dễ triển khai, phù hợp với các ứng dụng điều khiển và truyền nhận chuỗi ký tự như gửi lệnh, nhận trạng thái hoặc nhận tin nhắn. Trên ESP32, chế độ Bluetooth Serial thường được xây dựng dựa trên SPP, cho phép điện thoại hoặc thiết bị khác kết nối tới module và trao đổi dữ liệu mà không cần thiết kế giao thức quá phức tạp ở tầng thấp.

Trong phạm vi đề tài, SPP đóng vai trò cầu nối giữa điện thoại và ESP32 để truyền nội dung tin nhắn cũng như các lệnh điều khiển cơ bản. Nhờ đặc điểm gần với UART, việc lập trình xử lý dữ liệu trở nên trực quan, giảm độ phức tạp của hệ thống và vẫn đáp ứng tốt yêu cầu giao tiếp không dây ở khoảng cách ngắn [5].

2.4 Công nghệ WiFi

2.4.1 Chuẩn 802.11

WiFi là nhóm chuẩn mạng không dây thuộc họ IEEE 802.11, cho phép thiết bị trao đổi dữ liệu qua sóng vô tuyến thay cho kết nối có dây. Trên ESP32, khối WiFi hỗ trợ các chuẩn 802.11b/g/n trong băng tần 2.4 GHz, đáp ứng tốt các ứng dụng IoT cần kết nối mạng nội bộ hoặc Internet với chi phí phần cứng thấp.

Trong thực tế, các chuẩn 802.11b/g/n khác nhau chủ yếu ở tốc độ truyền, phương pháp điều chế và khả năng sử dụng băng thông. ESP32 có thể hoạt động ở chế độ Station để kết nối đến bộ phát WiFi, chế độ SoftAP để tự tạo điểm truy cập hoặc chế độ đồng thời Station/AP khi hệ thống vừa cần truy cập mạng vừa cần giao tiếp với thiết bị khác. Ngoài ra, bộ thu phát WiFi của ESP32 còn hỗ trợ các đặc tính như quét mạng, cơ chế bảo mật và cấu hình băng thông phù hợp với từng ứng dụng.

Đối với đề tài này, WiFi đóng vai trò là hạ tầng truyền dữ liệu từ ESP32 ra mạng Internet để gửi yêu cầu đến dịch vụ bên ngoài. Nhờ chuẩn 802.11, hệ thống có thể kết nối với bộ định tuyến phổ biến, nhận địa chỉ IP động và duy trì liên lạc với máy chủ mà không cần phần cứng truyền thông chuyên dụng bổ sung.

2.4.2 Kết nối Internet và HTTP Client

Sau khi kết nối thành công tới điểm truy cập WiFi, ESP32 có thể sử dụng ngăn xếp mạng TCP/IP để trao đổi dữ liệu với máy chủ trên Internet. Trong mô hình này, thiết bị thường chờ sự kiện đã nhận địa chỉ IP rồi mới khởi tạo các phiên giao tiếp tầng ứng dụng như HTTP hoặc HTTPS, nhờ đó việc gửi và nhận dữ liệu diễn ra ổn định hơn.

Trong hệ sinh thái ESP-IDF, thư viện `esp_http_client` cung cấp tập API chuyên dùng để tạo các yêu cầu HTTP/HTTPS. Khởi tạo cấu hình bằng `esp_http_client_init()`, thực hiện phiên truyền bằng `esp_http_client_perform()` và giải phóng tài nguyên bằng `esp_http_client_cleanup()`. Thư viện cũng hỗ trợ cơ chế callback sự kiện, cho phép chương trình theo dõi quá trình kết nối, gửi dữ liệu và nhận phản hồi từ máy chủ.



Khi cần truyền dữ liệu an toàn, ESP32 có thể sử dụng HTTPS thông qua lớp SSL/TLS tích hợp. Khi đó, ứng dụng cần cấu hình địa chỉ **https** và cung cấp thông tin xác thực chứng chỉ máy chủ nếu muốn kiểm tra tính hợp lệ của kết nối. Trong đề tài này, HTTP Client là thành phần quan trọng để ESP32 gửi yêu cầu đến dịch vụ mạng, nhận dữ liệu phản hồi và tích hợp các chức năng trực tuyến như truy vấn nội dung hoặc gọi dịch vụ chuyển văn bản thành giọng nói.

2.5 Màn hình TFT

2.5.1 Cấu tạo, nguyên lý hiển thị

Màn hình TFT LCD (Thin-Film-Transistor Liquid Crystal Display) có cấu trúc nhiều lớp, trong đó vật liệu tinh thể lỏng được đặt giữa hai tấm kính. Kết hợp với các lớp phân cực, lớp căn chỉnh và bộ lọc màu RGB, màn hình có thể kiểm soát lượng ánh sáng đi qua để tạo ra màu sắc hiển thị tại từng điểm ảnh.

Trong công nghệ ma trận chủ động, mỗi điểm ảnh con được điều khiển bởi một transistor màng mỏng và tụ điện lưu điện tích tạm thời. Nhờ đó, trạng thái của từng điểm ảnh được duy trì ổn định giữa các lần làm tươi, giúp hình ảnh hiển thị rõ nét hơn, tốc độ đáp ứng nhanh hơn và giảm hiện tượng nhòe so với các công nghệ thụ động.

TFT LCD không tự phát sáng như OLED mà cần nguồn sáng nền, thường là đèn LED trắng, để tạo ảnh hiển thị. Vì vậy, một module màn hình TFT hoàn chỉnh thường bao gồm tấm nền TFT LCD, hệ thống đèn nền LED và mạch điều khiển hiển thị. Chất lượng hiển thị của màn hình phụ thuộc vào các yếu tố như kích thước, độ phân giải, mức tiêu thụ công suất và chuẩn giao tiếp với vi điều khiển.

2.5.2 Điều khiển bằng thư viện TFT_eSPI

Trong môi trường Arduino, thư viện TFT_eSPI thường được sử dụng để điều khiển các màn hình TFT giao tiếp SPI như ILI9341 [1]. Thư viện cung cấp các hàm cơ bản để khởi tạo màn hình, thiết lập hướng hiển thị, vẽ điểm, đường thẳng, hình chữ nhật, hiển thị văn bản và tô màu vùng ảnh.

Nhờ tối ưu cho tốc độ truyền dữ liệu qua SPI, TFT_eSPI giúp ESP32 cập nhật nội dung hiển thị nhanh và thuận tiện hơn trong quá trình lập trình. Trong đề tài này, thư viện được dùng để hiển thị tin nhắn, trạng thái hoạt động của hệ thống và một số thành phần giao diện người dùng trên màn hình TFT.

Đặc biệt, thư viện hỗ trợ công nghệ **Smooth Font** (Font chữ mượt) dựa trên việc giải mã các mảng dữ liệu rasterized lưu trong bộ nhớ chương trình (PROGMEM). Khác với các font bitmap thông thường dễ bị vỡ hạt và giới hạn ở bảng mã ASCII, Smooth Font cho phép nhúng trực tiếp các bộ font mã hóa Unicode (như font Arial 24pt mã hóa trong tệp **arial24.h**). Nhờ đó, vi điều khiển ESP32 có thể truy xuất dữ liệu font từ bộ nhớ Flash theo thời gian thực và hiển thị các ký tự Tiếng Việt có dấu với độ sắc nét cao, chống răng cưa hiệu quả.

2.6 Quy trình xử lý âm thanh kỹ thuật số

2.6.1 Chuẩn nén âm thanh MP3 và Google TTS API

Để bản tin có thể phát ra giọng nói từ văn bản, hệ thống sử dụng công nghệ của Google Text-to-Speech (TTS). Dữ liệu văn bản được gửi lên máy chủ của Google và trả về dưới định dạng



MP3.

- **MP3 Compression:** Là định dạng nén dữ liệu âm thanh có mất mát (lossy), giúp giảm dung lượng tệp tin mà vẫn giữ được chất lượng giọng nói rõ ràng. Điều này rất quan trọng khi truyền tải qua mạng WiFi trên vi điều khiển có tài nguyên hạn chế như ESP32.
- **Google TTS:** Cung cấp giọng đọc tiếng Việt tự nhiên, giúp bản tin không chỉ hiển thị chữ mà còn có khả năng tương tác bằng âm thanh sống động.

2.6.2 Cơ chế giải mã và bộ nhớ đệm (Buffering)

Do tốc độ mạng không ổn định, hệ thống cần một cơ chế xử lý thông minh để âm thanh không bị ngắt quãng:

- **Buffering:** Hệ thống cấp phát một vùng nhớ đệm (thường từ 60KB đến 90KB) để lưu trữ một phần dữ liệu MP3 trước khi phát.
- **Software Decoding:** ESP32 sử dụng sức mạnh của hai lõi xử lý để giải mã tệp tin nén MP3 sang chuỗi dữ liệu thô (PCM) theo thời gian thực thông qua thư viện ESP8266Audio [6].

2.6.3 Mạch khuếch đại và đầu ra âm thanh

Dữ liệu sau khi giải mã được đẩy qua bộ điều khiển I2S tới mạch **MAX98357A**. Đây là mạch khuếch đại Class D tích hợp sẵn bộ giải mã DAC bên trong. Nó tiếp nhận tín hiệu số từ ESP32, chuyển đổi sang tín hiệu Analog và khuếch đại để phát trực tiếp ra loa mà không cần thêm các linh kiện lọc nhiễu phức tạp [4].

2.7 Bộ nhớ Flash (Preferences)

2.7.1 Lưu trữ dữ liệu không bay hơi

Trên ESP32, ngoài SRAM dùng để lưu dữ liệu tạm thời, hệ thống còn cần bộ nhớ không bay hơi để giữ lại thông tin sau khi mất nguồn hoặc khởi động lại. Nhu cầu này thường xuất hiện khi thiết bị phải lưu các tham số cấu hình như tên mạng WiFi, mật khẩu hoặc trạng thái hoạt động.

Trong môi trường Arduino, thư viện **Preferences** cho phép truy cập vùng nhớ flash không bay hơi theo cơ chế khóa-giá trị, được xây dựng trên nền NVS của ESP32. Thư viện hỗ trợ lưu và đọc nhiều kiểu dữ liệu thông dụng, giúp việc quản lý thông tin cấu hình trở nên đơn giản và phù hợp với các hệ thống nhúng có lượng dữ liệu nhỏ. Tuy nhiên, do flash có số lần ghi hữu hạn, chương trình cần tránh cập nhật liên tục không cần thiết.

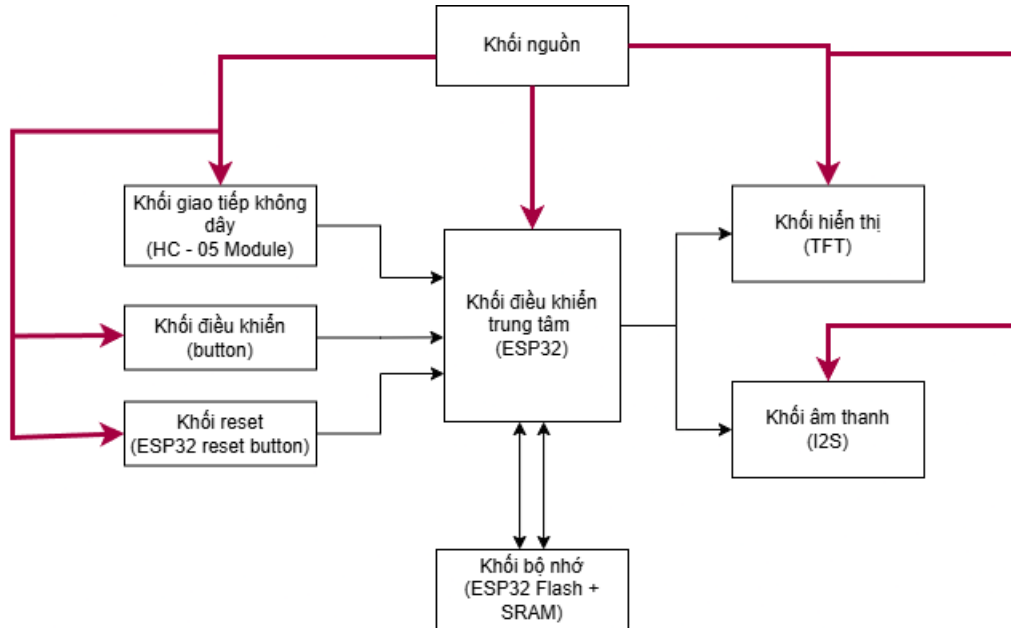
2.7.2 Quản lý tin nhắn trên ESP32

Trong thiết bị nhúng tin dùng ESP32, **Preferences** có thể được dùng để lưu một số dữ liệu cần duy trì như tin nhắn gần đây, vị trí hiển thị hiện tại hoặc các thiết lập giao diện và kết nối. Nhờ đó, khi hệ thống khởi động lại, thiết bị vẫn có thể khôi phục trạng thái sử dụng cơ bản.

Do dung lượng NVS không lớn, chương trình chỉ nên lưu những thông tin thật sự cần thiết và tổ chức dữ liệu theo từng khóa để dễ truy xuất, cập nhật và xóa. Cách lưu trữ này phù hợp với yêu cầu của đề tài vì vừa tiết kiệm tài nguyên, vừa giúp hệ thống hoạt động ổn định hơn.

3 CÁC PHẦN TỬ, LINH KIỆN TRONG ĐỒ ÁN

3.1 Sơ đồ khối



Hình 1: Sơ đồ khối của hệ thống

- **Khối nguồn:** Đảm nhiệm việc tiếp nhận và phân phối năng lượng cho toàn bộ hệ thống. Trong thiết kế nguyên mẫu này, hệ thống sử dụng một mạch nguồn ngoài cắm trực tiếp lên testboard (Breadboard Power Supply Module). Mạch này làm nhiệm vụ nhận điện áp từ Adapter DC hoặc cáp USB, sau đó ổn áp và cấp mức điện áp 5V và 3,3V trực tiếp cho các thanh ray (power rails) của testboard, từ đó cung cấp đủ dòng điện để nuôi vi điều khiển ESP32 và các module ngoại vi hoạt động ổn định.
- **Khối giao tiếp không dây:** sử dụng module Bluetooth HC-05 để thiết lập kết nối không dây giữa hệ thống và thiết bị bên ngoài, chẳng hạn như điện thoại thông minh.
 - Nhận dữ liệu tin nhắn từ thiết bị gửi và truyền về khối điều khiển trung tâm để xử lý.
 - Hỗ trợ giao tiếp không dây ở cự ly gần, giúp hệ thống thuận tiện hơn trong quá trình sử dụng.
- **Khối điều khiển:** dùng để chuyển chế độ hiển thị tin nhắn, kích hoạt chế độ *read-only* và xem các tin nhắn đã được lưu trong bộ nhớ flash.
- **Khối reset:** dùng để đặt lại module, giúp hệ thống hoạt động ổn định trong trường hợp bị nhiễu hoặc khi cần khởi động lại toàn bộ chương trình.
- **Khối điều khiển trung tâm:** sử dụng board ESP32 (đã được giới thiệu ở trên), đóng vai trò là “đầu não” của toàn bộ hệ thống.
 - Tiếp nhận dữ liệu từ khối giao tiếp không dây và tín hiệu điều khiển từ các nút nhấn.



- Xử lý nội dung tin nhắn, điều khiển hiển thị trên màn hình, phát âm thanh và quản lý dữ liệu lưu trữ.
- **Khối hiển thị:** sử dụng màn hình TFT ILI9341 để hiển thị màu sắc, cho phép hiển thị tin nhắn mới nhận tức thì hoặc các tin nhắn đã lưu trong bộ nhớ flash.
 - Hiển thị các thông tin trạng thái như số lượng tin nhắn đã lưu, chế độ *read-only* bật/tắt và hiệu ứng đang sử dụng.
- **Khối âm thanh:** sử dụng module MAX98357 để nhận tín hiệu số từ khối điều khiển trung tâm, sau đó khuếch đại và đưa âm thanh ra loa 3W.
 - Hỗ trợ phát thông báo âm thanh, góp phần tăng tính trực quan và khả năng tương tác của hệ thống.
- **Khối bộ nhớ:** sử dụng bộ nhớ tích hợp sẵn trên board ESP32 để lưu trữ dữ liệu phục vụ cho quá trình vận hành hệ thống.
 - Lưu các tin nhắn đã nhận để người dùng có thể xem lại khi cần.
 - Lưu một số thông tin cấu hình như chế độ hoạt động hoặc trạng thái sử dụng gần nhất của hệ thống.

3.2 DevBoard ESP32 WROOM 32U CP2102

3.2.1 Tổng quan

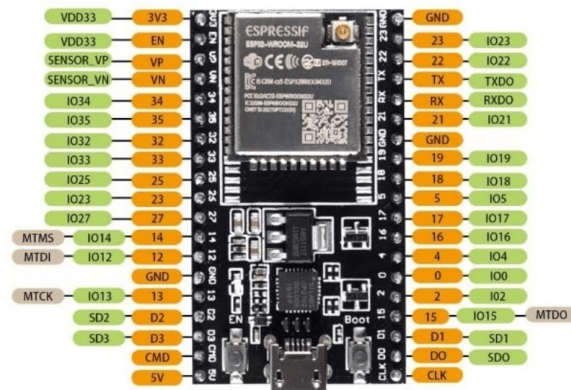
ESP32 - D0WDQ6 - V3 là vi điều khiển chính của đồ án, thuộc dòng MCU 32 - bit lõi kép, tích hợp sẵn WiFi và Bluetooth, phù hợp cho các ứng dụng nhúng và IoT. Bo mạch có bộ nhớ đủ cho việc lưu trữ và xử lý dữ liệu, hỗ trợ giao tiếp USB-UART qua CP2102 hoặc CH9102 để thuận tiện nạp chương trình. Ngoài ra, ESP32 hoạt động với nguồn 5 V hoặc 3,3 V, có kích thước nhỏ gọn và hỗ trợ nhiều môi trường lập trình như Arduino IDE, Mixly, Mind+ và Python.

3.2.2 Thông số kỹ thuật

- Mô hình bảng phát triển: ESP32-DevKitC-32
- Mô hình mô - đun: ESP32-WROOM-32
- Chip điều khiển chính: ESP32-D0WDQ6-V3 lõi kép 32bit MCU Tích hợp Wifi, Bluetooth
- Tích hợp 520-KB SRAM, 448-KB ROM, 16-KB SRAM in RTC
- Bộ nhớ ngoài: 4MB
- Chip điều khiển USB: CP2102
- Nguồn điện Vin Đầu vào điện áp 5V, hỗ trợ nguồn điện USB, nguồn điện 3.3V bên ngoài, và bộ nguồn Vin phát triển.
- Kích thước: loại USB - C: 52 * 28mm



3.2.3 Hình ảnh, mô tả chân



Hình 2: Sơ đồ chân của ESP32



Bảng 1: Mô tả các chân tín hiệu của ESP32

Tên chân tín hiệu	Mô tả
VIN	Chân cấp nguồn đầu vào 5 V cho bo mạch.
3V3	Chân cấp nguồn đầu ra 3,3 V, dùng cho các ngoại vi hoạt động ở mức điện áp thấp.
GND	Chân nối đất chung của toàn bộ hệ thống.
EN	Chân kích hoạt vi điều khiển, dùng để đặt lại hệ thống khi kéo xuống mức thấp.
VP/GPIO36	Chân ngõ vào analog, thường dùng để đọc tín hiệu từ cảm biến.
VN/GPIO39	Chân ngõ vào analog, hỗ trợ đo và nhận tín hiệu đầu vào.
GPIO34, GPIO35	Các chân chỉ hỗ trợ ngõ vào, thích hợp để nhận tín hiệu từ cảm biến hoặc nút nhấn.
GPIO32, GPIO33	Các chân GPIO đa dụng, hỗ trợ đọc/ghi tín hiệu số và chức năng ADC.
GPIO25, GPIO26	Các chân GPIO đa dụng, có thể sử dụng cho DAC, ADC hoặc điều khiển thiết bị ngoại vi.
GPIO27, GPIO14, GPIO12, GPIO13	Các chân GPIO thường được dùng cho giao tiếp SPI hoặc điều khiển tín hiệu số.
GPIO23	Chân MOSI mặc định của bus SPI, dùng để truyền dữ liệu đến thiết bị ngoại vi.
GPIO19	Chân MISO mặc định của bus SPI, dùng để nhận dữ liệu từ thiết bị ngoại vi.
GPIO18	Chân SCK mặc định của bus SPI, cung cấp xung nhịp cho quá trình truyền thông.
GPIO5	Chân thường dùng làm tín hiệu CS trong giao tiếp SPI.
GPIO21	Chân SDA, thường được sử dụng trong giao tiếp I2C.
GPIO22	Chân SCL, thường được sử dụng trong giao tiếp I2C.
GPIO1 (TX0)	Chân truyền dữ liệu UART, thường dùng để nạp chương trình và gỡ lỗi.
GPIO3 (RX0)	Chân nhận dữ liệu UART, thường dùng để nạp chương trình và gỡ lỗi.
GPIO17 (TX2), GPIO16 (RX2)	Các chân UART2, phù hợp để giao tiếp với các module bên ngoài như Bluetooth hoặc cảm biến.
GPIO4, GPIO2, GPIO15	Các chân GPIO đa dụng, có thể dùng để điều khiển LED, nút nhấn hoặc các ngoại vi số.
GPIO0	Chân lựa chọn chế độ khởi động, cần lưu ý khi sử dụng vì có ảnh hưởng đến quá trình nạp chương trình.

3.3 Module Bluetooth HC-05

3.3.1 Tổng quan

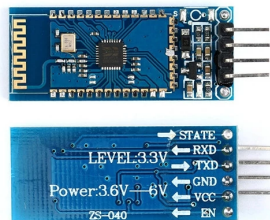
Module HC-05 là một thiết bị thu phát Bluetooth Classic (chuẩn V2.0+EDR) được thiết kế đặc biệt để thiết lập kết nối không dây nối tiếp trong suốt (Transparent Wireless Serial Connection). Khác với các module chỉ hoạt động ở chế độ Slave (như HC-06), HC-05 có khả năng hoạt động ở cả hai chế độ Master (chủ) và Slave (tớ). Trong khuôn khổ đồ án này, HC-05 được cấu hình ở chế độ Slave để lắng nghe, nhận luồng dữ liệu (chuỗi ký tự tin nhắn) từ điện thoại thông minh Android thông qua giao thức SPP (Serial Port Profile) [5] và truyền về vi điều khiển ESP32 qua giao tiếp UART.



3.3.2 Thông số kỹ thuật

- **Giao thức kết nối:** Bluetooth V2.0+EDR (Enhanced Data Rate).
- **Băng tần hoạt động:** 2.4 GHz ISM band.
- **Cấu hình hỗ trợ:** Bluetooth Serial Port Profile (SPP).
- **Giao tiếp phần cứng:** UART (Hỗ trợ baudrate từ 9600 đến 460800; mặc định thường dùng 9600 bps cho chế độ Data và 38400 bps cho chế độ AT Command).
- **Điện áp cấp nguồn (VCC):** 3,6 V đến 6 V DC (Tích hợp sẵn IC ổn áp trên board).
- **Điện áp logic giao tiếp (TX/RX):** 3,3 V TTL.
- **Dòng điện tiêu thụ:** Khoảng 30 mA khi đang chờ ghép nối (Pairing) và giảm xuống khoảng 10 mA khi đã thiết lập kết nối thành công.
- **Phạm vi kết nối tối đa:** Khoảng 10 mét (Thuộc nhóm thiết bị Class 2).

3.3.3 Hình ảnh, mô tả chân



Hình 3: Module Bluetooth HC-05



Tên chân	Mô tả chức năng
EN / KEY	Chân kích hoạt chế độ lệnh AT. Khi kéo lên mức cao (3,3V) trước khi cấp nguồn, module sẽ vào chế độ cấu hình (AT Command Mode) để đổi tên, mật khẩu hoặc baudrate.
VCC	Chân cấp nguồn dương (thường dùng nguồn 5V do module có sẵn IC ổn áp).
GND	Chân nối đất chung của hệ thống.
TXD	Chân truyền dữ liệu UART, nối với chân RX của vi điều khiển ESP32.
RXD	Chân nhận dữ liệu UART, nối với chân TX của vi điều khiển ESP32. <i>Lưu ý: Mức logic chân này là 3,3V, nếu dùng với VDK 5V (như Arduino Uno) cần qua cầu phân áp để tránh hỏng module, tuy nhiên ESP32 cũng dùng logic 3,3V nên có thể nối trực tiếp.</i>
STATE	Chân báo trạng thái kết nối (xuất tín hiệu cho LED). Nhấp nháy nhanh khi chờ kết nối, nhấp nháy chậm khi ở chế độ AT, và sáng/nhấp kép khi đã kết nối thành công.

Bảng 2: Mô tả các chân tín hiệu của module Bluetooth HC-05

3.4 Mạch khuếch đại âm thanh Max98357

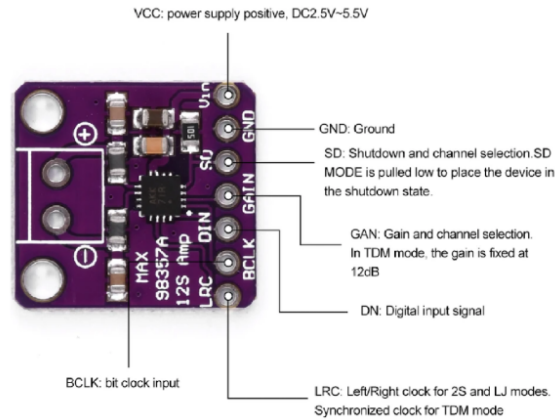
3.4.1 Tổng quan

Mạch khuếch đại âm thanh MAX98357 là một bộ khuếch đại công suất kỹ thuật số Class D với giao tiếp I2S (Inter-IC Sound). Nó chuyển đổi tín hiệu âm thanh kỹ thuật số sang tín hiệu analog và khuếch đại âm thanh trực tiếp đến loa. Điều này giúp loại bỏ nhu cầu về DAC (Digital-to-Analog Converter) riêng biệt trong nhiều ứng dụng âm thanh.

3.4.2 Thông số kỹ thuật

- Nguồn cung cấp: 2,5 V đến 5,5 V.
- Công suất đầu ra: 3,2 W/4 Ω ở mức 5 V.
- Dòng điện tĩnh: 2,4 mA.
- Hiệu suất: 92% ($R_L = 8 \Omega$, $P_{OUT} = 1 \text{ W}$).
- Tốc độ lấy mẫu: từ 8 kHz đến 96 kHz.
- PSRR: 77 dB ở 1 kHz.
- Tích hợp bảo vệ quá nhiệt và ngắn mạch.

3.4.3 Hình ảnh, mô tả chân



Hình 4: Sơ đồ chân của module khuếch đại Max98357

Bảng 3: Mô tả các chân tín hiệu của module MAX98357

Tên chân tín hiệu	Mô tả
LRC	Xung đồng bộ khung; là xung trái/phải trong chế độ I2S và LJ, đồng thời là xung đồng bộ trong chế độ TDM.
BCLK	Chân nhận xung nhịp bit.
DIN	Chân nhận tín hiệu dữ liệu số đầu vào.
GAIN	Chân thiết lập độ khuếch đại và lựa chọn kênh.
SD	Chân tắt nguồn và lựa chọn kênh; khi kéo chân SD_MODE xuống mức thấp, thiết bị sẽ chuyển sang chế độ tắt.
GND	Chân mass nguồn của mạch.
VCC	Chân cấp nguồn một chiều từ 2,5 V đến 5,5 V.

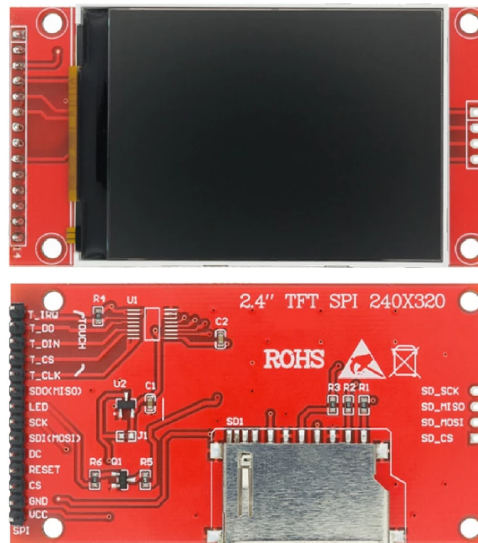
3.5 Loa 3W

3.6 Màn hình TFT

3.6.1 Tổng quan

Màn hình TFT sử dụng trong đồ án hoạt động với mức điện áp từ 3,3 V đến 5 V, tích hợp chip điều khiển ILI9341 và hỗ trợ độ phân giải 240×320 pixel. Màn hình hiển thị màu 18 bit, hỗ trợ tới 262.000 màu, cho chất lượng hiển thị rõ ràng và trực quan. Bên cạnh đó, màn hình sử dụng lớp cảm ứng điện trở, phù hợp cho các thao tác chạm cơ bản trong hệ thống. Đèn nền LED trắng giúp tăng khả năng quan sát nội dung hiển thị trong nhiều điều kiện ánh sáng khác nhau.

3.6.2 Hình ảnh, mô tả chân



Hình 5: Sơ đồ chân của màn hình TFT

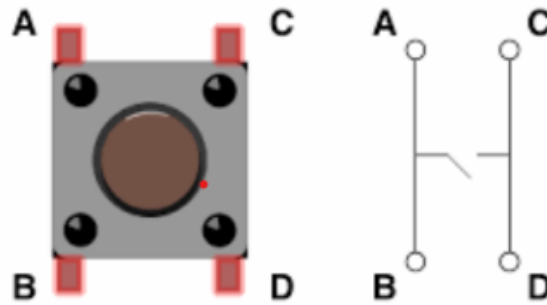
Bảng 4: Mô tả các chân tín hiệu của màn hình TFT

Tên chân tín hiệu	Mô tả
VCC	Nguồn điện đầu vào 5 V/3,3 V.
GND	Chân âm nguồn.
CS	Tín hiệu chọn chip LCD, kích hoạt ở mức thấp.
RESET	Tín hiệu đặt lại LCD, kích hoạt ở mức thấp.
DC	Tín hiệu chọn giữa thanh ghi LCD và dữ liệu; mức cao là thanh ghi, mức thấp là dữ liệu.
MOSI	Tín hiệu dữ liệu ghi của bus SPI.
SCK	Tín hiệu xung nhịp của bus SPI.
LED	Chân điều khiển đèn nền, bật ở mức cao.
MISO	Tín hiệu dữ liệu đọc của bus SPI.
T_CLK	Tín hiệu xung nhịp bus SPI của cảm ứng.
T_CS	Tín hiệu chọn chip cảm ứng, kích hoạt ở mức thấp.
T_DIN	Tín hiệu đầu vào bus SPI của cảm ứng.
T_DO	Tín hiệu đầu ra bus SPI của cảm ứng.
T_IRQ	Tín hiệu ngắt của màn hình cảm ứng, ở mức thấp khi phát hiện có chạm.



3.7 Button

3.7.1 Sơ đồ chân



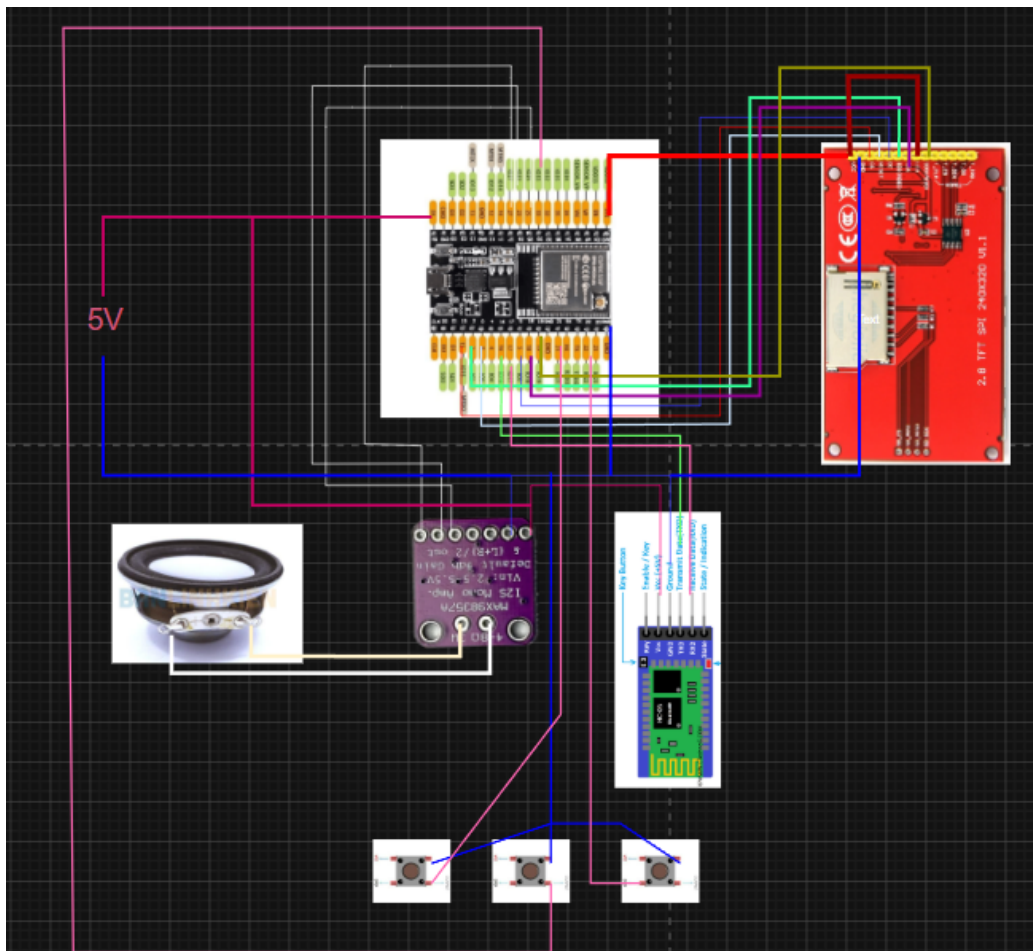
Hình 6: Sơ đồ chân của nút nhấn

Nguyên lí hoạt động:

- Ở trạng thái chưa nhấn: A nối với B, C nối với D.
- Ở trạng thái được nhấn: A, B, C, D sẽ được nối với nhau.

4 THIẾT KẾ HỆ THỐNG

4.1 Sơ đồ kết nối chân



Hình 7: Sơ đồ kết nối chân của hệ thống

4.2 Lưu đồ thuật toán

4.2.1 Các biến được khởi tạo

Để chương trình hoạt động ổn định và dễ quản lý, hệ thống khởi tạo một số biến dùng cho việc cấu hình phần cứng, xử lý giao diện, điều khiển âm thanh, lưu trữ dữ liệu và quản lý trạng thái hoạt động. Các biến này được trình bày trong các bảng sau.



Bảng 5: Các hằng số và biến cấu hình chính của hệ thống

Tên biến	Giá trị	Ý nghĩa
BUTTON_PIN	21	Chân nút bấm chính.
CLEAR_BUTTON	22	Chân nút xóa bộ nhớ.
DEBOUNCE_TIME	50	Thời gian chống rung, đơn vị ms.
LONG_PRESS_TIME	800	Thời gian nhấn dài để đổi chế độ hiển thị, đơn vị ms.
READ_ONLY_TIME	2000	Thời gian nhấn để khóa hoặc mở chế độ <i>read-only</i> , đơn vị ms.
I2S_DOUT	27	Chân dữ liệu I2S.
I2S_BCLK	26	Chân xung nhịp I2S.
I2S_LRC	25	Chân LRC của giao tiếp I2S.
BUTTON_SPEAK_PIN	32	Chân nút đọc tin nhắn.
RX2	16	Chân RX dùng cho module HC-05.
TX2	17	Chân TX dùng cho module HC-05.
messageCount	0	Số lượng tin nhắn hiện có.
currentMsgIdx	0	Chỉ số của tin nhắn đang hiển thị.
currentMode	0	Chế độ hiển thị hiện tại, nhận giá trị từ 0 đến 3.
xPos	0	Vị trí chạy chữ theo phương ngang.
msgWidth	0	Độ rộng của tin nhắn, đơn vị pixel.
speed	3	Tốc độ chạy chữ, đơn vị pixel trên mỗi lần cập nhật.



Bảng 6: Các biến thời gian của hệ thống

Tên biến	Giá trị	Ý nghĩa
buttonPressedTime	0	Thời điểm nút được nhấn để phục vụ việc đo thời gian nhấn.
lastBtnState	HIGH	Trạng thái trước đó của nút nhấn.

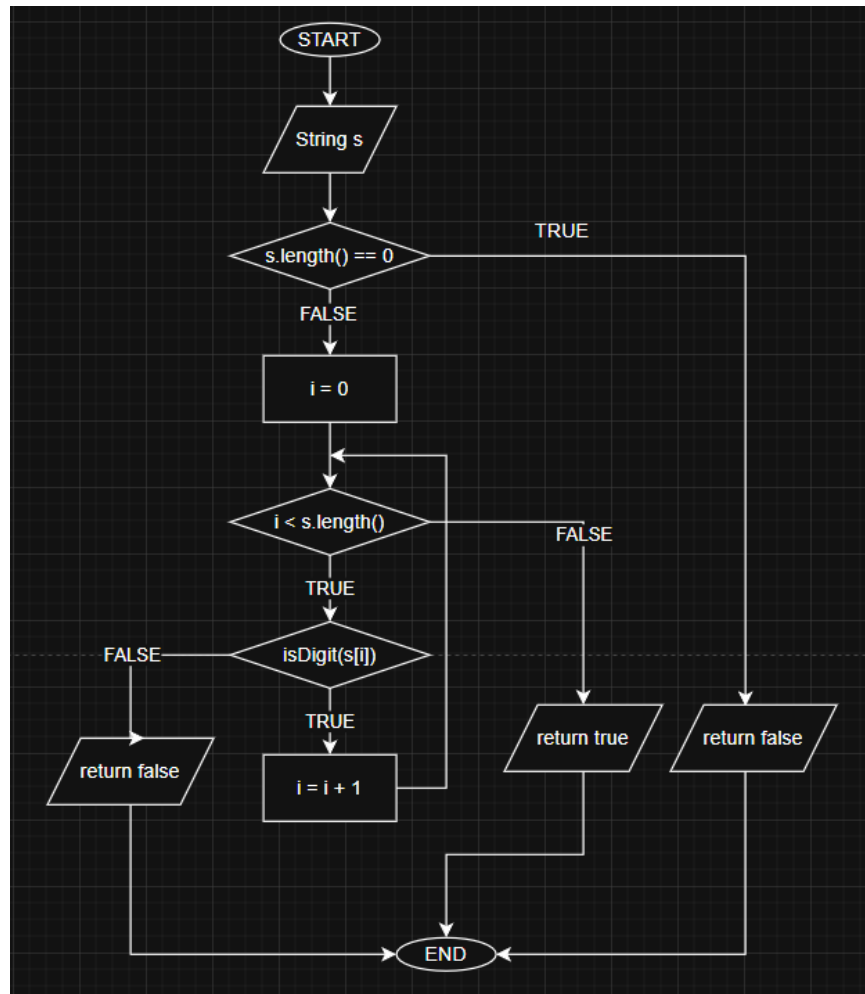
Bảng 7: Các biến logic và trạng thái của hệ thống

Tên biến	Giá trị	Ý nghĩa
lastBtnState	HIGH	Trạng thái nút ở lần kiểm tra trước.
isLongPressHandled	false	Cờ đánh dấu việc nhấn dài đã được xử lý hay chưa.
readOnlyMode	false	Trạng thái bật hoặc tắt chế độ chỉ đọc.
isProcessingBT	false	Cờ logic ngăn chặn việc đọc chồng chéo dữ liệu Bluetooth; chuyển sang true khi đang xử lý một chuỗi nhận được và trả về false khi hoàn thành.

Bảng 8: Các biến chuỗi và đối tượng được khởi tạo

Tên biến	Giá trị	Ý nghĩa
activeMsg	"Không có tin nhắn"	Tin nhắn mặc định hiển thị khi hệ thống chưa nhận được dữ liệu hoặc bộ nhớ trống.
ssid	"oii"	Tên mạng WiFi kết nối Internet để gọi Google TTS API.
password	"10122006"	Mật khẩu của mạng WiFi tương ứng.
out	AudioOutputI2S*	Con trỏ đối tượng xuất âm thanh qua giao tiếp I2S.
SerialBT	HardwareSerial(2)	Cổng UART dùng để giao tiếp với module HC-05.
tft	TFT_eSPI	Đối tượng điều khiển màn hình TFT.
pref	Preferences	Đối tượng quản lý bộ nhớ flash.
sprite	TFT_eSprite	Đối tượng sprite dùng để vẽ đệm trước khi hiển thị lên màn hình.

4.2.2 Hàm kiểm tra chuỗi có phải là số hay không (isNumber())

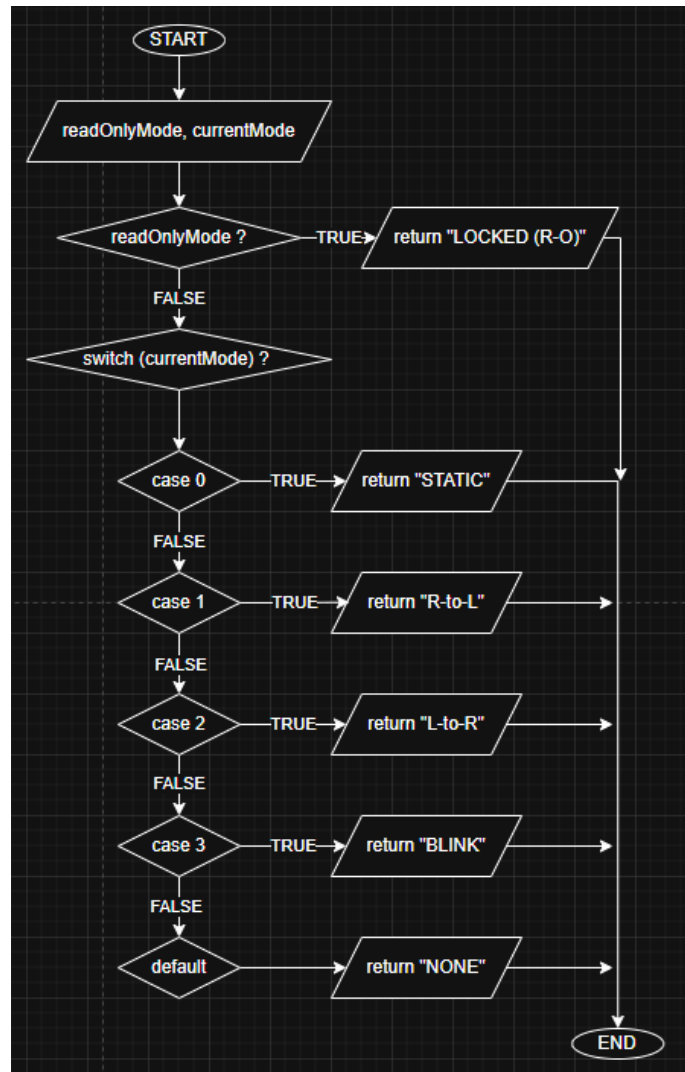


Hình 8: Lưu đồ thuật toán của hàm `isNumber()`

Chức năng:

- Kiểm tra một kí tự có phải là số hay không.
- Nếu độ dài chuỗi bằng 0 thì **return false**;
- Duyệt các kí tự trong string, nếu tồn tại 1 kí tự là chữ cái thì **return false**, không thì chuỗi đó toàn bộ là số, **return true**;

4.2.3 Hàm lấy tên chế độ hiển thị (getModeName())

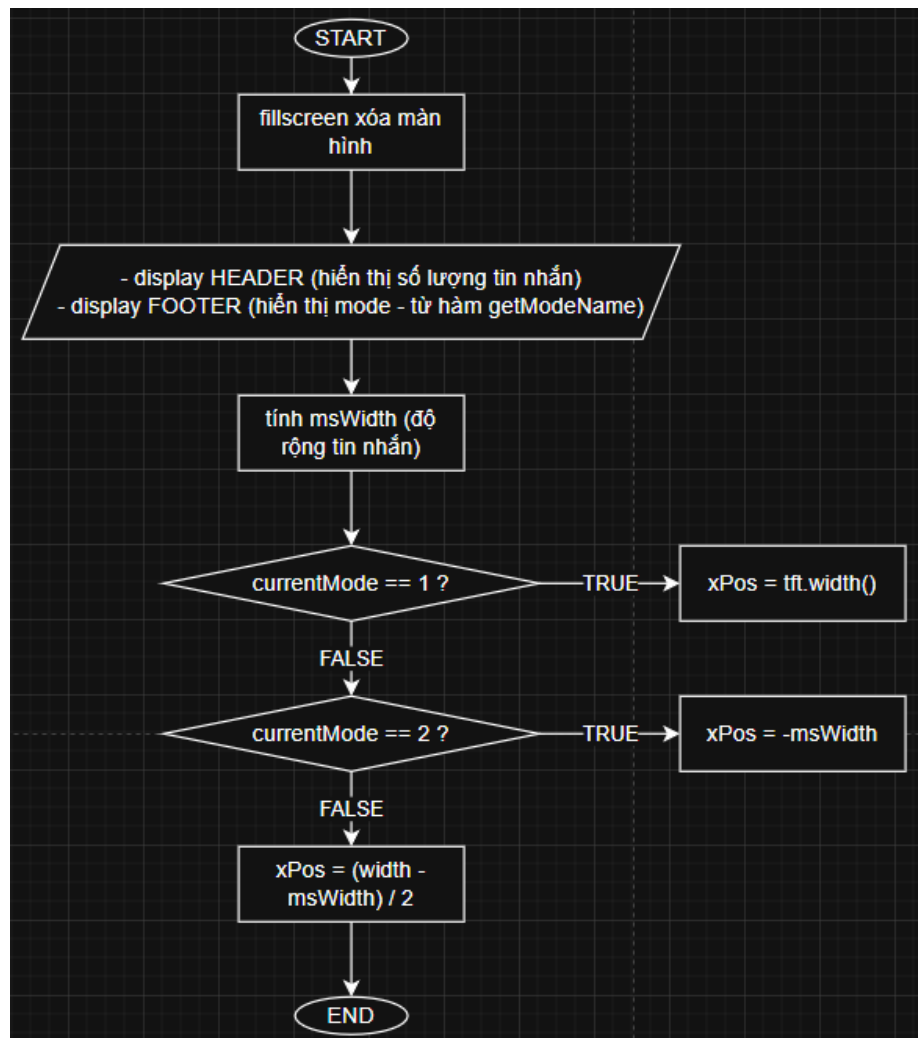


Hình 9: Lưu đồ thuật toán của hàm getModeName()

Chức năng:

- Lấy giá trị chế độ đang được hiển thị trên màn hình TFT.
- Gồm có 4 chế độ: STATIC (tĩnh), R - to - L (phải qua trái), L - to - R (trái qua phải), BLINK (nhấp nháy).
- Nếu không default sẽ là không có gì.

4.2.4 Hàm cập nhật giao diện hiển thị (updateUI())

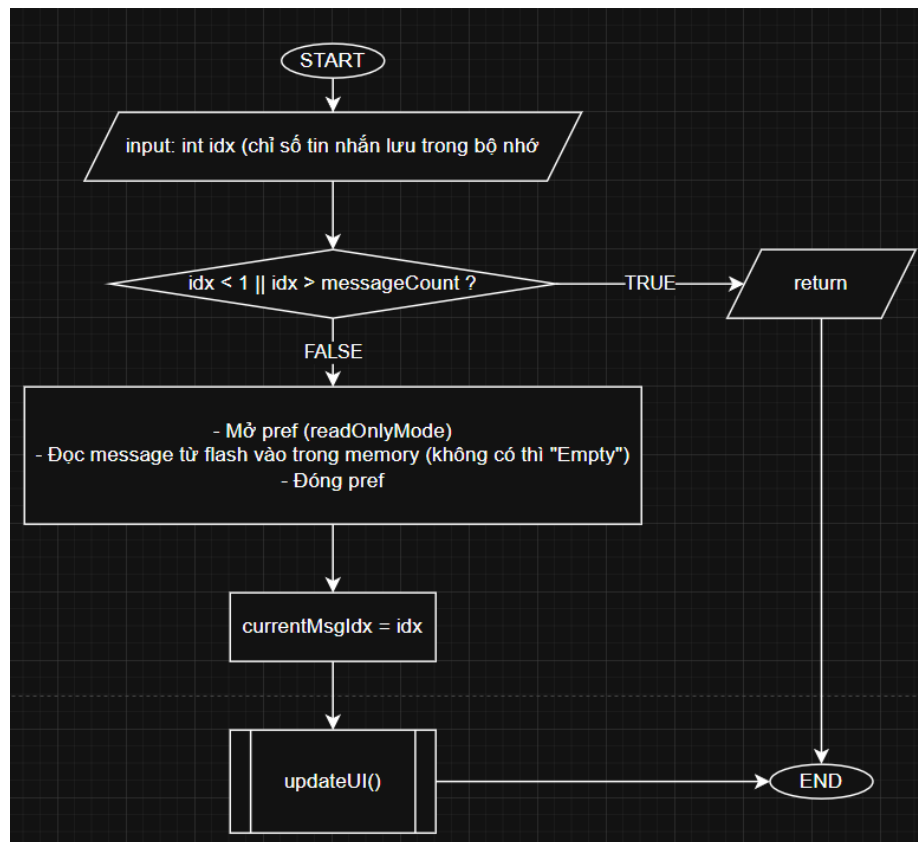


Hình 10: Lưu đồ thuật toán của hàm `updateUI()`

Chức năng:

- Hàm thực hiện xóa màn hình, thiết lập màu chữ, hiển thị số thứ tự tin nhắn ở phần header bằng chuỗi MSG: `currentMsgIdx/messageCount`.
- Hiển thị chế độ hiện tại ở footer bằng chuỗi MODE: `getModeName()`, tính độ rộng tin nhắn `msWidth` theo Smooth Font arial18, sau đó khởi tạo lại vị trí `xPos` theo hiệu ứng đang chọn.
- Tối ưu cho các hiệu ứng hiển thị chữ thông qua việc cập nhật lại các biến cục bộ.

4.2.5 Hàm tải tin nhắn từ bộ nhớ (loadMessage())

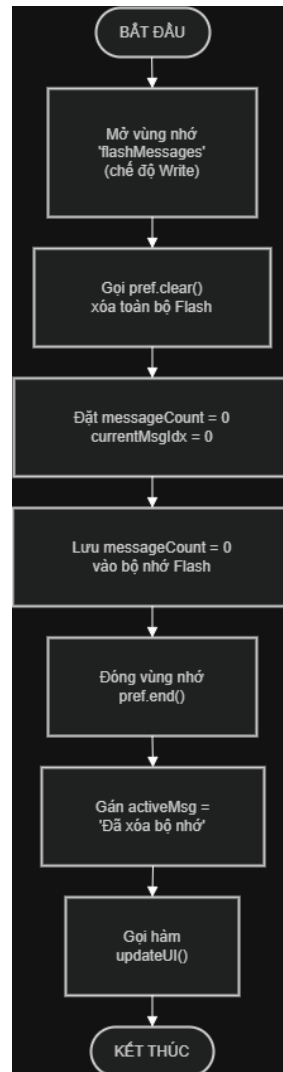


Hình 11: Lưu đồ thuật toán của hàm `loadMessage()`

Chức năng:

- Hàm mở phân vùng Preferences với tên định danh “flashMessages” ở chế độ chỉ đọc (read-only), truy xuất chuỗi tin nhắn theo khóa chỉ số kiểu `String(idx).c_str()`, cập nhật vào biến `activeMsg` rồi gọi `updateUI()`.
- Cập nhật lại `currentMsgIdx` để đồng bộ vị trí tin nhắn hiện tại với nội dung đang hiển thị trên màn hình.

4.2.6 Hàm xóa bộ nhớ flash (clearFlash())

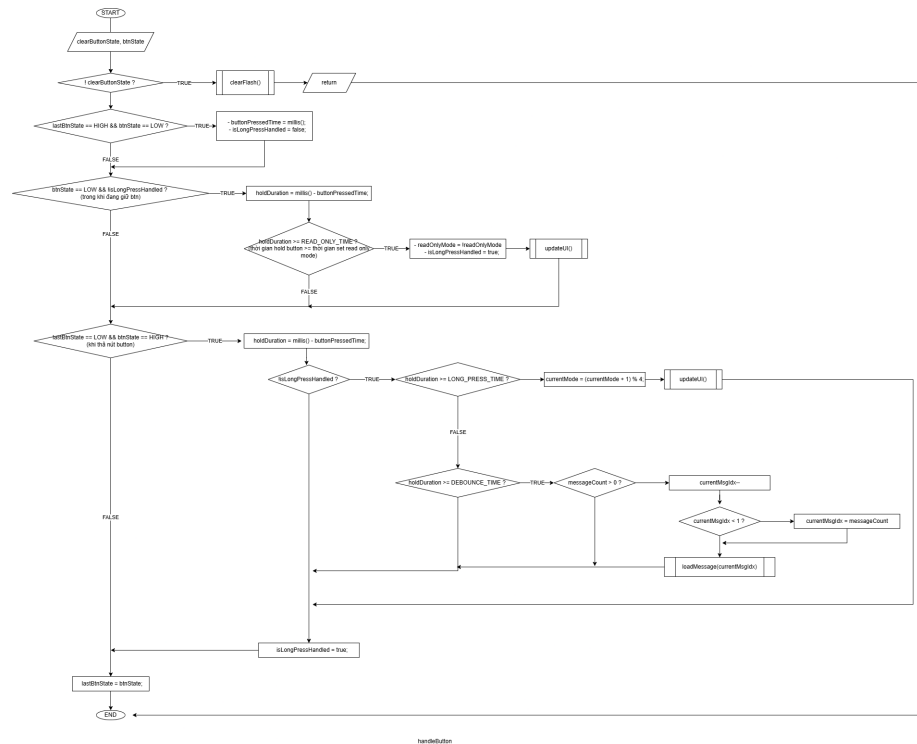


Hình 12: Lưu đồ thuật toán của hàm clearFlash()

Chức năng:

- Hàm mở phân vùng bộ nhớ và gọi lệnh `pref.clear()` để xóa sạch toàn bộ các cặp khóa-giá trị tin nhắn đã lưu, sau đó thiết lập `activeMsg` thành “Đã xóa bộ nhớ”.
- Reset các biến toàn cục như `messageCount` và `currentMsgIdx` để đưa hệ thống về trạng thái ban đầu.
- Cập nhật lại giao diện để hiển thị dữ liệu đúng sau khi xóa bộ nhớ Flash.

4.2.7 Hàm xử lý nút nhấn (handleButton())

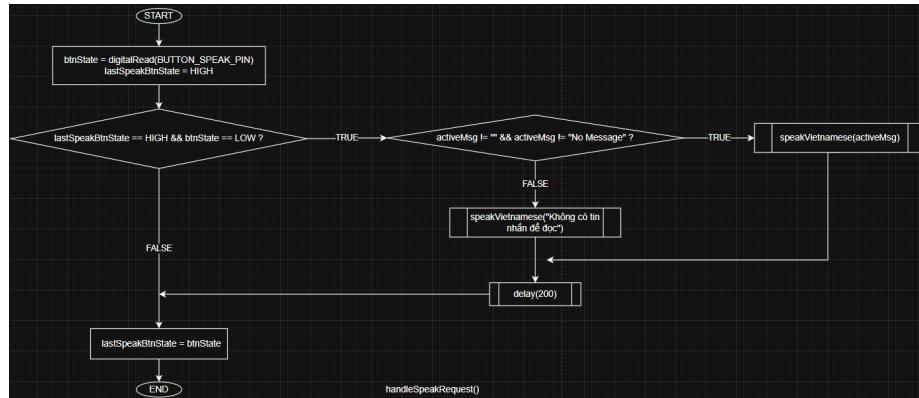


Hình 13: Lưu đồ thuật toán của hàm `handleButton()`

Chức năng:

- Handle để thực hiện tác vụ trong hệ thống.
- **Nhấn nhả nhanh:** xem lại tin nhắn cũ.
- **Nhấn giữ từ 0.8 giây đến dưới 2 giây:** chuyển lần lượt giữa các hiệu ứng hiển thị STATIC, R-to-L, L-to-R, BLINK.
- **Nhấn giữ (hơn 2 giây):** vào chế độ read - only.

4.2.8 Hàm xử lý yêu cầu phát âm thanh (handleSpeakRequest())

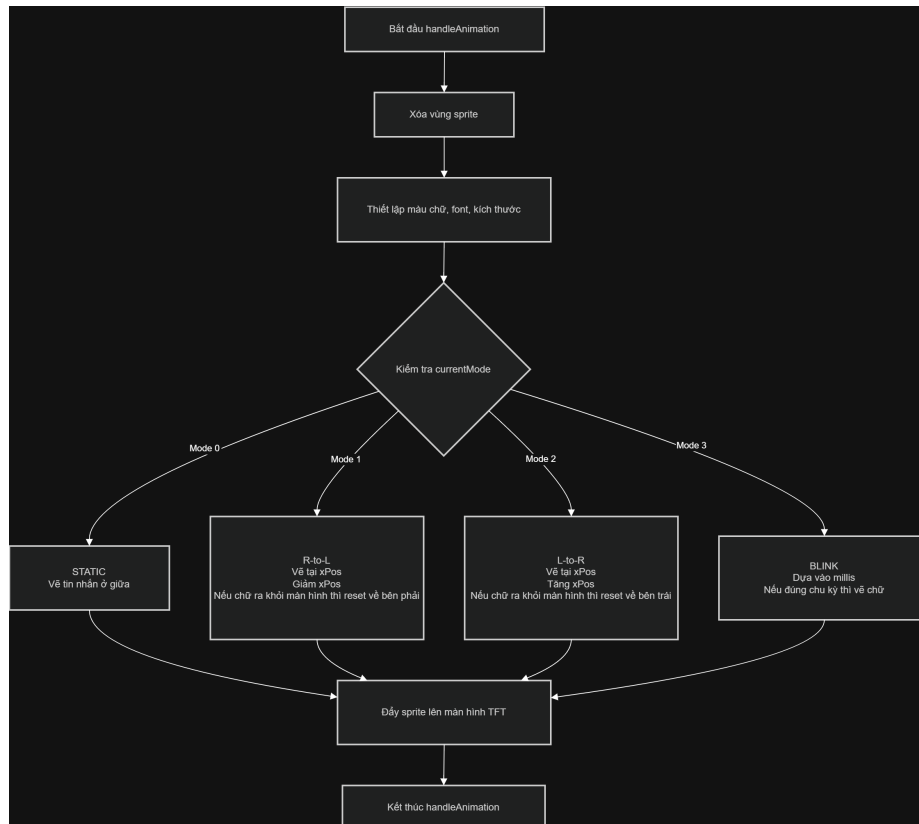


Hình 14: Lưu đồ thuật toán của hàm handleSpeakRequest()

Chức năng:

- Được dùng để đọc tin nhắn đang hiển thị trên màn hình.
- Nếu không có tin nhắn để đọc thì loa cần phát âm là: "Không có tin nhắn để đọc".

4.2.9 Hàm xử lý hiệu ứng hiển thị (handleAnimation())

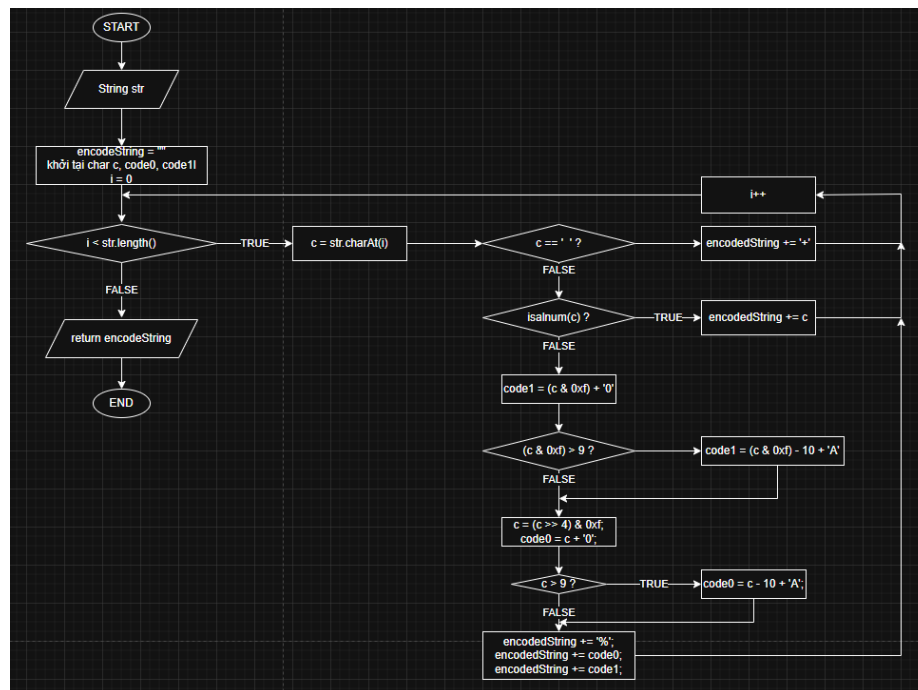


Hình 15: Lưu đồ thuật toán của hàm handleAnimation()

Chức năng:

- Xử lý, kiểm soát việc di chuyển của các kí tự khi hiển thị tin nhắn với từng hiệu ứng đã chọn.
- Như đã nói ở trên, có 4 chế độ, tương ứng với nó cần 4 case để handle hiệu ứng, cho ra chất lượng hiệu ứng mượt mà.
- Đẩy Sprite ra màn hình (vùng giữa màn hình) và đảm bảo tọa độ y không làm đè lên Header/Footer.

4.2.10 Hàm mã hóa chuỗi URL (urlencode())

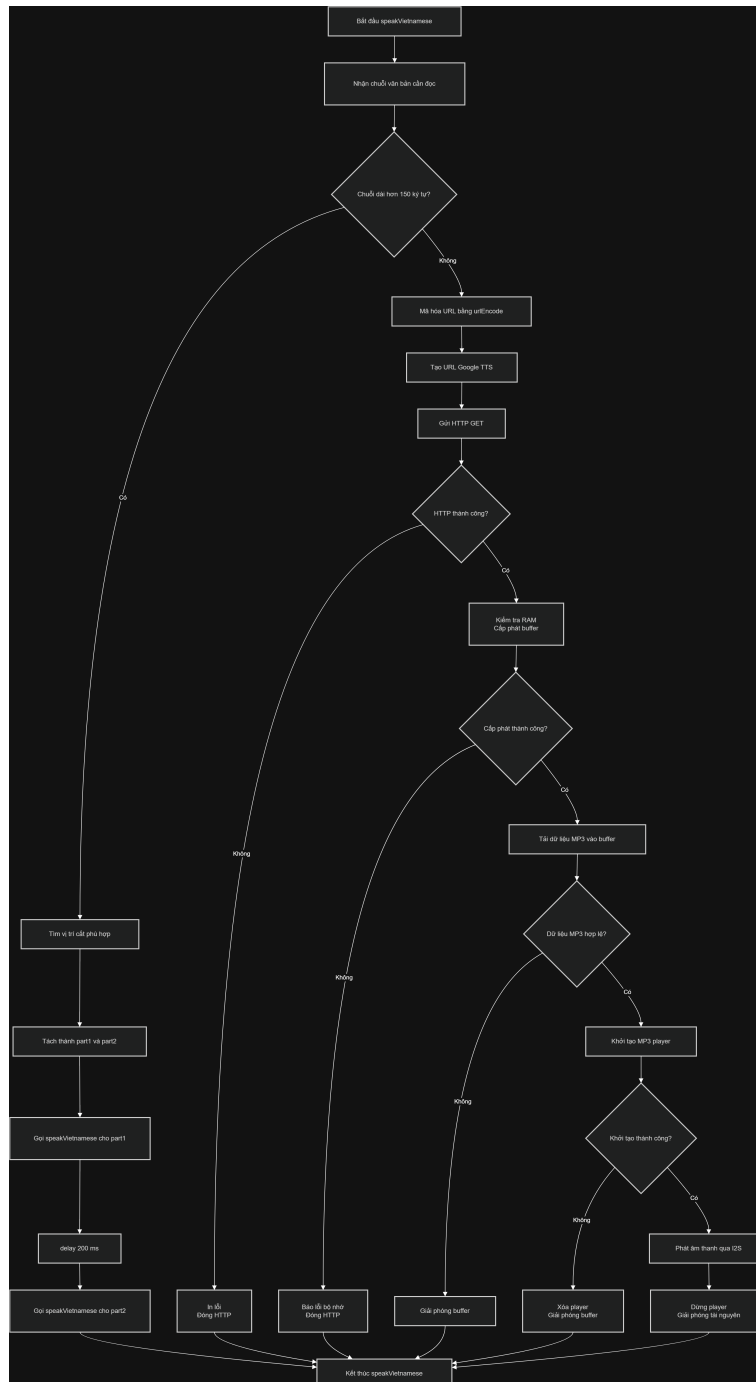


Hình 16: Lưu đồ thuật toán của hàm urlencode()

Chức năng:

- Chuẩn hóa dữ liệu để trình duyệt/server hiểu đúng.
- Hỗ trợ truyền tải Tiếng Việt không lỗi.
- Bảo vệ hệ thống khỏi các lỗi cú pháp URL.

4.2.11 Hàm phát tiếng Việt (speakVietnamese())



Hình 17: Lưu đồ thuật toán của hàm `speakVietnamese()`

Chức năng:

- Giảm ảnh hưởng của giới hạn độ dài văn bản bằng cách chia chuỗi dài thành nhiều đoạn nhỏ hơn trước khi gửi lên Google TTS.



4.3 Code và giải thích thuật toán

- Code interface:

```
#include <AudioGeneratorMP3.h>
#include <AudioOutputI2S.h>
#include <AudioFileSourcePROGMEM.h>
#include <HTTPClient.h>
#include <Preferences.h>
#include <TFT_eSPI.h>
#include <WiFi.h>
#include "arial18.h" // File font Vietnamese

// Define PIN and time
#define BUTTON_PIN 21
#define CLEAR_BUTTON 22
#define DEBOUNCE_TIME 50
#define LONG_PRESS_TIME 800 // 0.8s for changing mode
#define READ_ONLY_TIME 2000 // 2s for enable/disable Read-only mode

// Define for I2S
#define I2S_DOUT 27
#define I2S_BCLK 26
#define I2S_LRC 25
#define BUTTON_SPEAK_PIN 32

// Define UART for HC-05
#define RX2 16
#define TX2 17

// Initialize bluetooth, speaker, TFT
AudioOutputI2S *out;
HardwareSerial SerialBT(2); // Select UART 2
TFT_eSPI tft = TFT_eSPI();
Preferences pref;
TFT_eSprite sprite = TFT_eSprite(&tft);

// wifi's information
const char* ssid = "oii";
const char* password = "10122006";

// State variable
String activeMsg = "Không có tin nhắn";
int messageCount = 0;
int currentMsgIdx = 0;
int currentMode = 0;
int xPos = 0;
int msWidth = 0;
int speed = 3;

// Button control variables
unsigned long buttonPressedTime = 0;
bool lastBtnState = HIGH;
bool isLongPressHandled = false;
bool readOnlyMode = false;
bool isProcessingBT = false; // Bluetooth spam protection

bool isNumber(String s) {
    if (s.length() == 0) return false;
    for (int i = 0; i < s.length(); i++) {
        if (!isDigit(s[i])) return false;
    }
    return true;
}
```



```
String getModeName() {
    if (readOnlyMode) return "READ-ONLY MODE (R-0)";
    switch (currentMode) {
        case 0: return "STATIC";
        case 1: return "R-to-L";
        case 2: return "L-to-R";
        case 3: return "BLINK";
        default: return "NONE";
    }
}

void updateUI() {
    tft.fillScreen(TFT_BLACK);
    tft.setTextSize(1);

    // Drawing Header
    tft.setTextColor(TFT_CYAN, TFT_BLACK);
    tft.setTextDatum(TL_DATUM);
    tft.drawString("MSG: " + String(currentMsgIdx) + "/" + String(messageCount), 5, 5, 0);

    // Drawing Footer
    tft.setTextColor(readOnlyMode ? TFT_RED : TFT_MAGENTA, TFT_BLACK);
    tft.setTextDatum(BL_DATUM);
    tft.drawString("MODE: " + getModeName(), 5, tft.height() - 5, 0);

    // Accurately calculate message width using Arial16 font (Font index = 0)
    msWidth = tft.textWidth(activeMsg, 0);

    // Reset text position at startup
    if (currentMode == 1) xPos = tft.width();
    else if (currentMode == 2) xPos = -msWidth;
    else xPos = (tft.width() - msWidth) / 2;
}

void loadMessage(int idx) {
    if (idx < 1 || idx > messageCount) return;
    pref.begin("flashMessages", true);
    activeMsg = pref.getString(String(idx).c_str(), "Trống");
    pref.end();
    currentMsgIdx = idx;
    updateUI();
}

void clearFlash() {
    pref.begin("flashMessages", false);
    pref.clear();
    pref.putInt("msgCount", 0);
    pref.end();

    messageCount = 0;
    currentMsgIdx = 0;
    activeMsg = "Đã xóa bộ nhớ";
    updateUI();
    delay(500);
}

void handleButton() {
    bool clearButtonState = digitalRead(CLEAR_BUTTON);
    bool btnState = digitalRead(BUTTON_PIN);
    unsigned long holdDuration = 0;

    // Clear message's memory
    if (!clearButtonState) {
        clearFlash();
    }
}
```



```
    return;
}

// Pressing button
if (lastBtnState == HIGH && btnState == LOW) {
    buttonPressedTime = millis();
    isLongPressHandled = false;
}

// Checking hold button
if (btnState == LOW && !isLongPressHandled) {
    holdDuration = millis() - buttonPressedTime;
    // Checking hold duration is enough for R-O mode
    if (holdDuration >= READ_ONLY_TIME) {
        readOnlyMode = !readOnlyMode;
        isLongPressHandled = true;
        updateUI();
    }
}

// Release button
if (lastBtnState == LOW && btnState == HIGH) {
    holdDuration = millis() - buttonPressedTime;
    if (!isLongPressHandled) {
        // Checking hold duration is enough for text animation
        if (holdDuration >= LONG_PRESS_TIME) {
            currentMode = (currentMode + 1) % 4;
            updateUI();
        }
        // Displaying old messages
        else if (holdDuration >= DEBOUNCE_TIME) {
            if (messageCount > 0) {
                currentMsgIdx--;
                if (currentMsgIdx < 1)
                    currentMsgIdx = messageCount;
                loadMessage(currentMsgIdx);
            }
        }
    }
    isLongPressHandled = true;
}
lastBtnState = btnState;
}

void handleSpeakRequest() {
    static bool lastSpeakBtnState = HIGH;
    bool btnState = digitalRead(BUTTON_SPEAK_PIN);

    if (lastSpeakBtnState == HIGH && btnState == LOW) {
        if (activeMsg != "" && activeMsg != "Không có tin nhắn") {
            speakVietnamese(activeMsg);
        } else {
            speakVietnamese("Không có tin nhắn để đọc");
        }
        delay(200);
    }
    lastSpeakBtnState = btnState;
}

void handleAnimation() {
    sprite.fillSprite(TFT_BLACK);
    sprite.setTextColor(TFT_YELLOW, TFT_BLACK);
    sprite.setTextSize(1);

    int fontIdx = 0;
```



```
switch (currentMode) {
    case 0: // Static (Central)
        sprite.setTextDatum(MC_DATUM);
        sprite.drawString(activeMsg, tft.width() / 2, sprite.height() / 2, fontIdx);
        break;

    case 1: // Right to Left
        sprite.setTextDatum(TL_DATUM);
        sprite.drawString(activeMsg, xPos, 10, fontIdx);
        xPos -= speed;
        if (xPos < -msWidth) {
            xPos = tft.width();
        }
        break;

    case 2: // Left to Right
        sprite.setTextDatum(TL_DATUM);
        sprite.drawString(activeMsg, xPos, 10, fontIdx);
        xPos += speed;
        if (xPos > tft.width()) {
            xPos = -msWidth;
        }
        break;

    case 3: // Blink
        if ((millis() / 500) % 2 == 0) {
            sprite.setTextDatum(MC_DATUM);
            sprite.drawString(activeMsg, tft.width() / 2, sprite.height() / 2, fontIdx);
        }
        break;
}
sprite.pushSprite(0, (tft.height() / 2) - 20);
}

String urlEncode(String str) {
    String encodedString = "";
    char c;
    char code0;
    char code1;

    for (int i = 0; i < str.length(); i++) {
        c = str.charAt(i);
        if (c == ' ') {
            encodedString += '+';
        } else if (isalnum(c)) {
            encodedString += c;
        } else {
            code1 = (c & 0xf) + '0';
            if ((c & 0xf) > 9) {
                code1 = (c & 0xf) - 10 + 'A';
            }
            c = (c >> 4) & 0xf;
            code0 = c + '0';
            if (c > 9) {
                code0 = c - 10 + 'A';
            }
            encodedString += '%';
            encodedString += code0;
            encodedString += code1;
        }
    }
    return encodedString;
}
```




```
void speakVietnamese(String text) {
    Serial.printf("\nĐọc: %s\n", text.c_str());
    // Google TTS limit ~200 char
    if (text.length() > 150) {
        Serial.println(" Text dài, chia nhỏ...");

        int cutPos = 150;
        while (cutPos > 50 && text.charAt(cutPos) != ' ' && text.charAt(cutPos) != ',') {
            cutPos--;
        }

        String part1 = text.substring(0, cutPos);
        String part2 = text.substring(cutPos + 1);

        speakVietnamese(part1);
        delay(200);
        speakVietnamese(part2);
        return;
    }

    String encodedText = urlEncode(text);
    Serial.printf("Encode text: ", encodedText);
    String url = "http://translate.google.com/translate_tts?ie=UTF-8&tl=vi&client=tw-ob&q=" + encodedText;

    // ===== DOWNLOAD MP3 =====
    HTTPClient http;
    http.setTimeout(10000);
    http.begin(url);
    http.addHeader("User-Agent", "Mozilla/5.0");

    int httpCode = http.GET();

    if (httpCode != 200) {
        Serial.printf(" HTTP error: %d\n", httpCode);
        http.end();
        return;
    }

    const size_t MAX_BUFFER = 60000; // 60KB max

    // ===== CHECKING RAM BEFORE MALLOC =====
    if (ESP.getFreeHeap() < MAX_BUFFER + 10000) {
        Serial.printf(" Không đủ RAM! Cần %d, có %d\n", MAX_BUFFER + 10000, ESP.getFreeHeap());
        http.end();
        return;
    }

    // Allocate buffer
    uint8_t* buffer = (uint8_t*)malloc(MAX_BUFFER);
    if (!buffer) {
        Serial.println(" malloc failed!");
        http.end();
        return;
    }

    // Download with timeout
    WiFiClient* stream = http.getStreamPtr();
    size_t downloaded = 0;
    unsigned long startTime = millis();
    unsigned long lastDataTime = millis();

    while (http.connected() && downloaded < MAX_BUFFER) {
        size_t available = stream->available();

        if (available) {
```



```
int toRead = min(available, MAX_BUFFER - downloaded);
int bytesRead = stream->readBytes(buffer + downloaded, toRead);
downloaded += bytesRead;
lastDataTime = millis(); // Reset timeout when have data
}

// Timeout: 2s without new data -> done
if (millis() - lastDataTime > 2000) {
    Serial.println(" Download xong!");
    break;
}

// Total timeout: 10s
if (millis() - startTime > 10000) {
    Serial.println(" Download timeout!");
    break;
}

yield();
delay(1);
}

http.end();

if (downloaded < 100) {
    Serial.println(" File quá nhỏ, bỏ qua");
    free(buffer);
    return;
}

// ===== PLAY FROM BUFFER =====
AudioFileSourcePROGMEM *audioSrc = new AudioFileSourcePROGMEM(buffer, downloaded);
AudioGeneratorMP3 *player = new AudioGeneratorMP3();

if (!player->begin(audioSrc, out)) {
    Serial.println(" MP3 begin failed!");
    delete audioSrc;
    delete player;
    free(buffer);
    return;
}

Serial.println("Đang phát...");

// Play with timeout
unsigned long playStart = millis();

while (player->isRunning()) {
    if (!player->loop()) {
        break;
    }

    // Timeout playing: 15s
    if (millis() - playStart > 15000) {
        Serial.println("Play timeout!");
        break;
    }

    yield();
}

// Cleanup
player->stop();
delete audioSrc;
delete player;
```



```
free(buffer);

Serial.println("Phát xong!");
delay(10);
}

void setup() {
  Serial.begin(115200);
  SerialBT.begin(9600, SERIAL_8N1, RX2, TX2);

  pinMode(CLEAR_BUTTON, INPUT_PULLUP);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(BUTTON_SPEAK_PIN, INPUT_PULLUP);

  tft.init();
  tft.setRotation(1);
  tft.fillScreen(TFT_BLACK);

  // ===== LOADING FONT ARRAY INTO TFT & SPRITE =====
  tft.loadFont(arial18);
  sprite.loadFont(arial18);

  sprite.setColorDepth(16);
  sprite.createSprite(tft.width(), 40);

  tft.setTextColor(TFT_WHITE, TFT_BLACK);
  tft.setTextDatum(MC_DATUM);
  tft.drawString("Đang kết nối WiFi...", tft.width()/2, tft.height()/2, 0);

  pref.begin("flashMessages", true);
  messageCount = pref.getInt("msgCount", 0);
  pref.end();

  WiFi.mode(WIFI_STA);
  WiFi.disconnect(true);
  delay(100);
  WiFi.begin(ssid, password);

  unsigned long wifiStart = millis();
  while (WiFi.status() != WL_CONNECTED) {
    if (millis() - wifiStart > 15000) {
      tft.fillScreen(TFT_BLACK);
      tft.setTextColor(TFT_RED, TFT_BLACK);
      tft.setTextDatum(MC_DATUM);
      tft.drawString("Lỗi kết nối WiFi!", tft.width()/2, tft.height()/2, 0);
      delay(1500);
      break;
    }
    delay(200);
  }

  out = new AudioOutputI2S();
  out->SetPinout(I2S_BCLK, I2S_LRC, I2S_DOUT);

  if (messageCount > 0) loadMessage(messageCount);
  else updateUI();

  if (WiFi.status() == WL_CONNECTED) {
    speakVietnamese("Thiết bị sẵn sàng nhận tin nhắn");
  }
  Serial.println("Hệ thống sẵn sàng");
}

void loop() {
  handleButton();
}
```



```
handleSpeakRequest();

// Processing Bluetooth's data (prevent spam + support Vietnamese text clean)
if (SerialBT.available() && !isProcessingBT) {
    isProcessingBT = true;

    String input = SerialBT.readStringUntil('\n');
    input.trim();

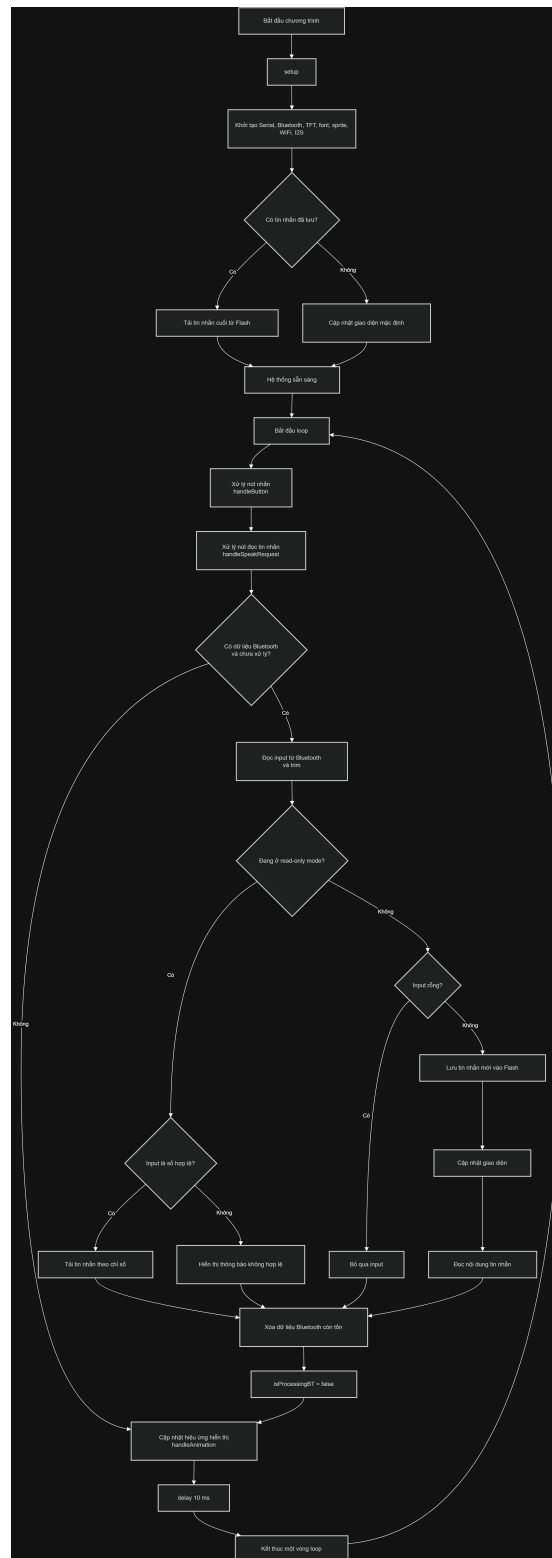
    if (readOnlyMode) {
        if (isNumber(input)) {
            int idx = input.toInt();
            if (idx >= 1 && idx <= messageCount) {
                loadMessage(idx);
            } else {
                activeMsg = "Không tìm thấy!";
                updateUI();
            }
        }
    }
    else if (input.length() > 0) {
        speakVietnamese("Bạn có tin nhắn mới");
        pref.begin("flashMessages", false);
        messageCount++;
        pref.putString(String(messageCount).c_str(), input);
        pref.putInt("msgCount", messageCount);
        pref.end();

        activeMsg = input;
        currentMsgIdx = messageCount;
        updateUI();
        speakVietnamese(activeMsg);
    }

    // Clear stale buffer data while audio is playing
    while (SerialBT.available() > 0) {
        SerialBT.read();
    }
    isProcessingBT = false;
}

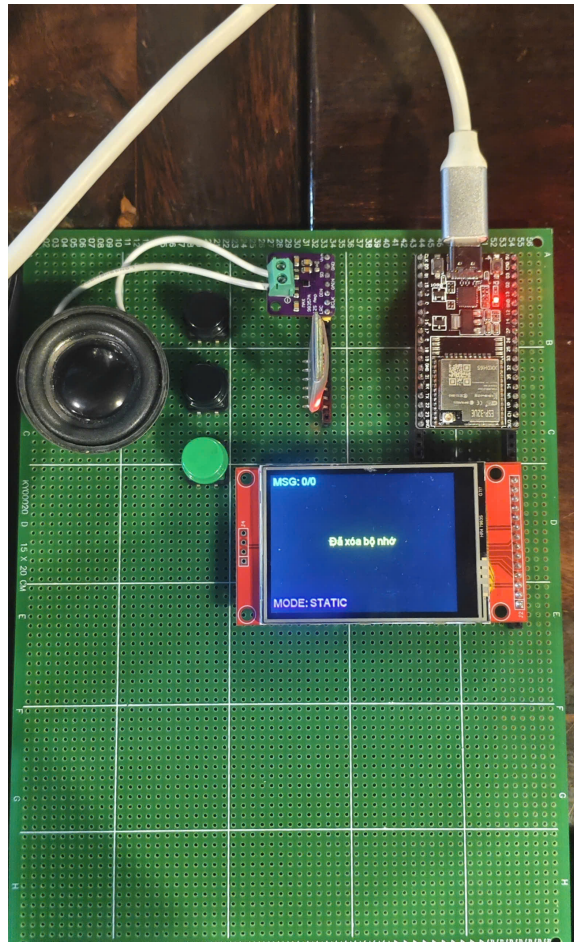
handleAnimation();
delay(10);
}
```

- Flowchart tổng quát:



Hình 18: Lưu đồ thuật toán tổng quát của hệ thống

4.4 Hiện thực phần cứng (video, hình ảnh)



Hình 19: Hình ảnh mạch thực tế



5 KẾT LUẬN

5.1 Kết quả đã đạt được:

Sau quá trình nghiên cứu và thực nghiệm, nhóm đã hoàn thành mục tiêu xây dựng hệ thống bảng tin Bluetooth di động với các kết quả cụ thể như sau:

Về mặt phần cứng:

- Thiết kế và lắp ráp thành công hệ thống phần cứng hoàn chỉnh với **ESP32-WROOM-32U** làm trung tâm điều khiển.
- Tối ưu hóa hiển thị trên màn hình **TFT ILI9341 (240x320 pixels)** qua giao tiếp SPI, đảm bảo màu sắc 18-bit rõ nét và tốc độ phản hồi cao.
- Tích hợp module **Bluetooth HC-05** và mạch khuếch đại âm thanh **MAX98357** sử dụng giao thức **I2S**, kết hợp loa 3W cho âm thanh to, rõ ràng và không bị nhiễu.

Về mặt phần mềm và thuật toán:

- Hiện thực hóa thành công giải pháp nhúng *Smooth Font* lưu trong **PROGMEM**, giải quyết triệt để bài toán hiển thị Tiếng Việt có dấu trên màn hình TFT LCD mà không làm tiêu tốn bộ nhớ RAM động (SRAM).
- Xây dựng cơ chế lọc và xử lý chuỗi nối tiếp từ Bluetooth qua cờ hiệu **isProcessingBT**, giúp hệ thống vận hành ổn định, hạn chế treo và bỏ qua dữ liệu tồn đọng (chống spam).
- Đồng bộ hóa hoàn toàn giao diện người dùng (UI) và hệ thống phát âm thanh (TTS) bằng ngôn ngữ Tiếng Việt, mang lại trải nghiệm tối ưu cho mô hình bảng tin thông báo di động.

5.2 Hạn chế:

Mặc dù hệ thống đã đáp ứng được các yêu cầu cơ bản và hoạt động ổn định, song do giới hạn về mặt phần cứng và đặc thù của các linh kiện được sử dụng, đồ án vẫn còn tồn tại một số hạn chế nhất định:

- **Giới hạn về khoảng cách và khả năng tương thích của Bluetooth:** Do sử dụng module HC-05 giao tiếp qua chuẩn Bluetooth Classic (giao thức SPP), phạm vi kết nối ổn định chỉ nằm trong bán kính dưới 10m (không có vật cản). Thêm vào đó, giao thức SPP chủ yếu tương thích tốt với các thiết bị Android, trong khi gặp khó khăn hoặc không thể kết nối trực tiếp với các thiết bị iOS nếu không có chứng chỉ MFi (Made for iPhone).
- **Sự phụ thuộc vào kết nối Internet của tính năng TTS:** Hệ thống được định vị là “bảng tin Bluetooth di động”, tuy nhiên tính năng chuyển đổi văn bản thành giọng nói (Text-to-Speech) lại yêu cầu vi điều khiển ESP32 phải kết nối WiFi để gọi API từ Google. Nếu hoạt động ở môi trường ngoại tuyến (không có WiFi/4G hotspot), tính năng phát âm thanh sẽ bị vô hiệu hóa.
- **Giới hạn bộ nhớ RAM khi xử lý âm thanh:** Do vi điều khiển ESP32 có dung lượng SRAM hạn chế, việc tải và giải mã trực tiếp luồng MP3 từ Google TTS vào RAM (thông qua



cấp phát động `malloc` với giới hạn buffer khoảng 60KB) khiến hệ thống không thể xử lý các câu văn quá dài trong một lần gọi. Hiện tại mã nguồn đang phải chia nhỏ các câu lớn hơn 150 ký tự, đôi khi làm giảm tính liên mạch của giọng đọc.

- **Khả năng hiển thị và tiêu thụ năng lượng:** Màn hình TFT ILI9341 có kích thước khá nhỏ (2.4–2.8 inch), chỉ phù hợp cho việc quan sát ở khoảng cách gần. Đồng thời, việc phải duy trì liên tục kết nối Bluetooth, module WiFi (cho TTS), cùng lúc với mạch khuếch đại âm thanh và độ sáng màn hình làm tăng đáng kể mức tiêu thụ dòng điện, khiến hệ thống cần nguồn cấp ổn định từ cổng USB hoặc các thiết bị cấp nguồn di động (như sạc dự phòng) để duy trì hoạt động lâu dài.

5.3 Hướng phát triển:

Thay vì nâng cấp phần cứng làm tăng giá thành thiết bị, nhóm nghiên cứu đề xuất các giải pháp cải tiến chuyên sâu về mặt mã nguồn và thuật toán nhằm khai thác tối đa hiệu năng của các linh kiện hiện tại:

- **Tối ưu hóa giao tiếp không dây (Loại bỏ module HC-05):** Thay vì sử dụng linh kiện rời, mã nguồn có thể được viết lại để khai thác trực tiếp vi mạch Bluetooth/BLE tích hợp sẵn trên ESP32 (thông qua thư viện `BluetoothSerial.h` hoặc các thư viện BLE). Giải pháp này giúp cắt giảm chi phí, thu gọn kích thước mạch phần cứng, tiết kiệm năng lượng và đặc biệt là hỗ trợ kết nối được với hệ điều hành iOS (nếu sử dụng chuẩn BLE).
- **Cải tiến thuật toán đọc âm thanh bằng kỹ thuật Streaming:** Hiện tại, hệ thống đang dùng `malloc` để tải toàn bộ tệp MP3 từ Google TTS vào RAM trước khi phát (bị giới hạn buffer 60KB). Hướng phát triển tới là thay đổi kiến trúc code, sử dụng lớp `AudioFileSourceHTTPStream` của thư viện `ESP8266Audio`. Kỹ thuật này cho phép nhận dữ liệu đến đâu, giải mã và phát ra loa đến đó (On-the-fly decoding), giúp vi điều khiển đọc được các đoạn văn bản dài vô hạn mà không bao giờ bị tràn bộ nhớ.
- **Phát triển ứng dụng điều khiển chuyên dụng (Mobile App):** Xây dựng một ứng dụng di động độc lập (sử dụng các nền tảng cross-platform như Flutter hoặc React Native) thay vì dùng các Bluetooth Terminal chung chung. Ứng dụng này sẽ cung cấp giao diện trực quan để người dùng thiết lập màu sắc màn hình, điều chỉnh tốc độ chạy chữ hoặc cài đặt lịch phát âm thanh.



TÀI LIỆU THAM KHẢO

- [1] Bodmer. TFT_eSPI: Arduino and PlatformIO IDE compatible TFT library. https://github.com/Bodmer/TFT_eSPI, 2023.
- [2] Espressif Systems. *ESP32 Technical Reference Manual*. Espressif Systems, 2024. https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf.
- [3] Wiley E Fossett. *ESP32 S3/C3/C6 Development Guide: Practical IoT Programming with ESP-IDF, Arduino, WiFi, BLE, and Embedded Systems Design*. Amazon Digital Services LLC, 2026.
- [4] Maxim Integrated. *MAX98357A Datasheet: PCM Input Class D Audio Power Amplifiers*, 2020. Standard I2S Audio interface document.
- [5] Brent A Miller and Chatschik Bisdikian. *Bluetooth Revealed: The Insider's Guide to an Open Specification for Global Wireless Communications*. Prentice Hall PTR, 2000. Tài liệu chuẩn mực về giao thức SPP (Serial Port Profile) sử dụng trên module HC-05.
- [6] Earle F. Philhower. ESP8266Audio: Arduino library for parsing and decoding MP3 files and playing them on an I2S DAC. <https://github.com/earlephilhower/ESP8266Audio>, 2023.